

COURSE BOOK

Testing with Python

unittest, pytest and automotive software — a hands-on
course in 13 labs

Unit · Integration · System · Property-based · Robustness · Regression · Performance · Simulation · Mutation ·
Acceptance

Python 3.10+ · pytest · unittest · Hypothesis · built 07 October 2026

Contents

About this course

Lab 1 — Unit testing with `unittest`

Lab 2 — pytest fundamentals

Lab 3 — Black-box test design

Lab 4 — Test doubles: isolating the unit

Lab 5 — Integration testing

Lab 6 — State-transition and system-level testing

Lab 7 — Property-based testing with Hypothesis

Lab 8 — Robustness: negative testing, fault injection and fuzzing

Lab 9 — Regression, golden-file and log-replay testing

Lab 10 — Performance and real-time budget testing

Lab 11 — Software-in-the-loop (SIL) simulation and acceptance criteria

Lab 12 — Test adequacy: coverage and mutation testing

Lab 13 — Acceptance tests, requirements traceability and SIL/HIL switching

Capstone — Tyre-Pressure Monitoring bug hunt

Appendix A — Cheat sheet

Appendix B — Code under test

Appendix C — Course tooling

About this course

A hands-on course in which you learn **every major kind of software testing** by testing realistic automotive code (CAN bus, battery management, cruise control, ABS, diagnostics, tyre-pressure monitoring) with Python's `unittest` and `pytest`.

- 13 labs + a capstone "bug hunt" (≈ 35 contact hours)
- Every lab: concept → fully worked, runnable examples → exercises → debrief questions
- A verified instructor solution for every exercise (`solutions/`), executed in CI
- No hardware needed: ECUs, sensors and vehicle dynamics are simulated; a switch shows how the same tests would run on a real bench (HIL)

Quick start

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

make fast          # the everyday loop (≈ 3 s)
make lab N=06      # run one lab verbosely
pytest labs/lab02_pytest/exercise_lab02.py  # work on an exercise (fails until you finish it)
```

Windows without `make` : run the commands from the Makefile directly (`python -m pytest ...`).

Course map

#	Lab	Testing type / technique	Main tools	Automotive case
1	unittest fundamentals	Unit testing	<code>unittest.TestCase</code> , <code>subTest</code> , <code>setUp</code> , <code>assertRaises</code>	CAN frames, CRC-8, unit conversions, wheel-speed maths
2	pytest fundamentals	Unit testing, fixtures, data-driven	<code>assert</code> , <code>parametrize</code> , fixtures, <code>tmp_path</code> , markers	Temperature sensor, BMS fault detection
3	Black-box test design	Equivalence classes, boundary values, decision tables	<code>parametrize</code> , invariants	Battery charge-current derating
4	Test doubles	Isolation : stub, fake, spy, mock, dummy	<code>unittest.mock</code> , <code>monkeypatch</code> , <code>FakeClock</code>	ADC hardware, diagnostics, timeouts without sleeping
5	Integration testing	Integration / interface / contract	virtual bus, fault hooks	Wheel-speed ECU → CAN → instrument cluster
6	State machines & system tests	State-transition, functional/system, scenario	exhaustive sequences	Cruise-control modes

#	Lab	Testing type / technique	Main tools	Automotive case
7	Property-based testing	Property-based / generative	Hypothesis	Signal codec round-trips, E2E protection, BMS safety invariants
8	Robustness & fault injection	Negative, fault-injection, fuzz, recovery	seeded RNG, Hypothesis	Stuck sensors, bit errors, burst loss, fuzzing the UDS handler
9	Regression & golden files	Regression, snapshot/golden, log replay	custom <code>golden</code> fixture	DTC reports, UDS transcripts, recorded drive cycle
10	Performance & real-time budgets	Performance, timing, memory	<code>perf_counter</code> , <code>tracemalloc</code>	10 ms control cycle, saturated 500 kbit/s bus
11	Simulation (SIL)	Closed-loop / model-based, acceptance criteria	plant models	Cruise control on hills, ABS vs locked wheels
12	Coverage & mutation	Test-adequacy: coverage, mutation testing	<code>pytest-cov</code> , <code>tools/mini_mutate.py</code>	"100 % covered" suite that catches nothing
13	Acceptance, traceability, HIL	Acceptance (Given/When/Then), requirements traceability, SIL ↔ HIL	custom marker + report	ISO 26262-style requirement → test matrix
★	Capstone: TPMS bug hunt	Everything	all of the above	Tyre-pressure monitoring; graded against 17 hidden defects

Which test types appear where?

Test type	Labs	Test type	Labs
Unit	1, 2	Fuzz / negative	7, 8
Smoke / sanity	2, 5	Fault injection / recovery	5, 8
Black-box design	3	Regression / golden / replay	9
White-box (state, internals)	6, 12	Performance / memory	10
Integration / contract	5	Simulation (SIL) / HIL	11, 13
System / functional	6, 11	Test adequacy (coverage, mutation)	12
Property-based	7	Acceptance / traceability	13

Repository layout

src/autotest/	the software under test (the "ECU code")		
can.py	frames, signals, CRC-8, E2E protection	bms.py	battery management
sensors.py	conversions + plausibility-checked sensor	cruise.py	cruise state machine + PI
control			
bus.py	virtual CAN bus with fault injection	abs.py	ABS controller + quarter-
car plant			
ecu.py	wheel-speed ECU + instrument cluster	vehicle.py	longitudinal vehicle
model			
diagnostics.py	DTC debouncing + UDS-lite	tpms.py	capstone SUT
clock.py	SystemClock / FakeClock		
labs/labNN_*/	README.md · worked tests (test_*.py) · exercise_labNN.py		
capstone/	TPMS specification, starter suite, defect-injection grader		
solutions/	instructor solutions (they run in CI, so they cannot rot)		
tools/	mini_mutate.py (mutation tester), make_drive_log.py		
docs/	requirements.csv, cheat sheet, instructor guide		
conf/test.py	shared fixtures (clock, bus, golden), --hil / --update-golden / --req-report		

How each lab is organised

1. **Read** the lab README (10 min) — the concept and why it matters in a vehicle programme.
2. **Run and read** the worked tests (`test_labNN_*.py`) — they are deliberately commented teaching material.
3. **Break something** — mutate the code under test and watch which test fails (this builds intuition for Lab 12 early).
4. **Do the exercises** in `exercise_labNN.py` (not collected by default; run the file explicitly).
5. **Debrief** with the discussion questions.

Reference

- pdf/ — the whole course as a printable **Course Book** (learners) and **Instructor Edition**; rebuild with `make pdf`
- docs/cheatsheet.md — unittest ↔ pytest side-by-side, useful flags, marker recipes
- docs/requirements.csv — the requirement set used for traceability

Safety note. The code here is teaching material, not production automotive software. The models are simplified; never use them for real vehicle decisions.

Lab 1 — Unit testing with `unittest`

Duration: 2.5 h · **Type:** unit testing · **Code under test:** `can.py` , `sensors.py` , `bms.py`

Why it matters in a vehicle programme

A modern car runs 100M+ lines of code; the cheapest place to find a defect is the unit that contains it. A wrong CAN scale factor or a CRC off-by-one is invisible on the bench but costs a recall in the field. `unittest` ships with Python, so it works in locked-down toolchains where installing packages is hard.

Learning objectives

After this lab you can: write a `TestCase` ; choose the right `assert*` method; test exceptions and floating-point results; share set-up safely (`setUp` , `setUpClass`); run many inputs with `subTest` ; document skipped and known-failing tests; run tests from the CLI, as a hand-built suite, or through `pytest`.

Concepts (10 min)

- **Anatomy:** Arrange → Act → Assert. A unit test fails for exactly one reason.
- **Isolation:** `setUp` runs before *every* test, so tests cannot depend on each other.
- `subTest` : reports each failing input separately instead of stopping at the first.
- **Never `assertEqual` floats** — use `assertAlmostEqual` .
- `expectedFailure` / `skip` make known problems visible instead of deleting tests.

Run it

```
PYTHONPATH=src python -m unittest discover -s labs/lab01_unittest -p "test_lab01*.py" -v
pytest labs/lab01_unittest -v                # same tests, pytest runner
```

Guided tour of `test_lab01_unittest.py`

Class	Look for
<code>TestUnitConversions</code>	<code>subTest</code> , <code>assertAlmostEqual</code> , <code>assertRaisesRegex</code> — also the hand-computed expected value (14.4 km/h)
<code>TestCrc8</code>	A <i>catalogue check value</i> (0x4B) is the best oracle you can get; single-bit-flip loop proves CRC strength
<code>TestCANFrame</code>	<code>setUp</code> / <code>tearDown</code> ; frozen dataclass; boundary 0x7FF/0x800
<code>TestSignalScaling</code>	<code>setUpClass</code> , <code>@skip</code> , <code>@expectedFailure</code> documenting a known resolution defect

Class	Look for
<code>load_tests</code>	building a suite by hand

Try this (5 min): in `can.py` change `& 0xFF` in `crc8` to `& 0x7F`. Which tests fail, and which do *not*? What does that tell you about the suite?

Exercises — `exercise_lab01.py`

1. Coulomb counting in `BatteryManagementSystem.update_soc`: clamping, discharge, invalid input.
2. Encode/decode a two-signal engine message — work out the expected bytes **by hand first**.
3. E2E protection: round trip, corruption, counter wrap.

Debrief questions

- Why is `assertTrue(a == b)` a worse choice than `assertEqual(a, b)`?
- A test uses `setUpClass` to create a battery object and mutates it. What can go wrong?
- When is an `expectedFailure` better than a skipped test? Than a deleted test?

Common pitfalls

Tests that depend on execution order · asserting on `repr` /exception text that may legitimately change · catching exceptions manually (`try/except` hides "no exception" passing silently) · testing the implementation instead of behaviour.

Worked example — source

labs/lab01_unittest/test_lab01_unittest.py

```

"""Lab 1 - unit testing with the standard library's unittest.

Run: python -m unittest discover -s labs/lab01_unittest -p "test_lab01*.py" -v
or: pytest labs/lab01_unittest -v      (pytest runs unittest tests too)
"""
import unittest

from autotest.can import CANFrame, Signal, crc8
from autotest.sensors import celsius_to_fahrenheit, kmh_to_ms, ms_to_kmh, wheel_speed_kmh

class TestUnitConversions(unittest.TestCase):
    """Anatomy of a test: Arrange - Act - Assert."""

    def test_kmh_to_ms_known_value(self):
        self.assertAlmostEqual(kmh_to_ms(36.0), 10.0)

    def test_round_trip(self):
        for kmh in (0, 1, 50, 130.5):
            with self.subTest(kmh=kmh): # one failure does not hide the others
                self.assertAlmostEqual(ms_to_kmh(kmh_to_ms(kmh)), kmh)

    def test_freezing_point(self):

```

```

        self.assertEqual(celsius_to_fahrenheit(0), 32)

    def test_wheel_speed_formula(self):
        # 40 pulses / 20 per rev = 2 rev in 1 s; 2 m circumference -> 4 m/s = 14.4 km/h
        self.assertAlmostEqual(wheel_speed_kmh(40, 20, 2.0, 1.0), 14.4)

    def test_invalid_interval_raises(self):
        with self.assertRaises(ValueError):
            wheel_speed_kmh(10, 20, 2.0, 0)

    def test_error_message_is_helpful(self):
        with self.assertRaisesRegex(ValueError, "invalid wheel speed"):
            wheel_speed_kmh(-1, 20, 2.0, 1.0)

class TestCrc8(unittest.TestCase):
    def test_standard_check_value(self):
        # CRC-8/SAE-J1850 catalogue check value for ASCII "123456789"
        self.assertEqual(crc8(b"123456789"), 0x4B)

    def test_detects_single_bit_flip(self):
        good = bytes([0x12, 0x34, 0x56])
        for bit in range(24):
            with self.subTest(bit=bit):
                bad = bytearray(good)
                bad[bit // 8] ^= 1 << (bit % 8)
                self.assertNotEqual(crc8(good), crc8(bytes(bad)))

class TestCANFrame(unittest.TestCase):
    """Fixtures: setUp runs before EVERY test, so tests never share state."""

    def setUp(self):
        self.frame = CANFrame(0x123, b"\x01\x02\x03")

    def tearDown(self):
        pass # release files / sockets here; nothing needed for pure objects

    def test_dlc_is_data_length(self):
        self.assertEqual(self.frame.dlc, 3)

    def test_frame_is_immutable(self):
        with self.assertRaises(Exception):
            self.frame.arbitration_id = 0x7FF # frozen dataclass

    def test_standard_id_limit(self):
        CANFrame(0x7FF) # last legal id: must not raise
        with self.assertRaises(ValueError):
            CANFrame(0x800)

    def test_extended_id_accepts_29_bits(self):
        self.assertEqual(CANFrame(0x1FFFFFFF, is_extended=True).arbitration_id, 0x1FFFFFFF)

    def test_too_many_bytes(self):
        with self.assertRaises(ValueError):
            CANFrame(0x1, bytes(9))

class TestSignalScaling(unittest.TestCase):
    @classmethod
    def setUpClass(cls): # expensive shared setup, done once per class
        cls.rpm = Signal("rpm", 0, 16, factor=0.25, minimum=0, maximum=16000)

    def test_scaling(self):
        self.assertEqual(self.rpm.to_raw(3000), 12000)
        self.assertEqual(self.rpm.to_physical(12000), 3000)

    def test_out_of_range_rejected(self):

```

```

        with self.assertRaises(ValueError):
            self.rpm.to_raw(16001)

@unittest.skip("demonstration: skipped tests are reported, never silently lost")
def test_skipped(self):
    self.fail("never runs")

@unittest.expectedFailure
def test_known_bug_documented(self):
    # An expected failure documents a KNOWN defect; it turns red when the bug is fixed.
    self.assertEqual(self.rpm.to_raw(0.1), 1) # rounds to 0 - resolution is 0.25 rpm

def load_tests(loader, tests, pattern):
    """Optional hook: build a suite by hand (see README, section 'Suites and runners')."""
    suite = unittest.TestSuite()
    for case in (TestUnitConversions, TestCrc8, TestCANFrame, TestSignalScaling):
        suite.addTests(loader.loadTestsFromTestCase(case))
    return suite

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

Exercise skeleton — source

labs/lab01_unittest/exercise_lab01.py

```

"""Lab 1 exercises - complete every TODO, then run:
    pytest labs/lab01_unittest/exercise_lab01.py -v
"""
import unittest

from autotest.can import CANMessage, Signal, e2e_check, e2e_protect, E2EStatus
from autotest.bms import BatteryManagementSystem

class TestCoulombCounting(unittest.TestCase):
    """Exercise 1: test BatteryManagementSystem.update_soc."""

    def setUp(self):
        self.bms = BatteryManagementSystem(capacity_ah=50.0, soc=50.0)

    def test_charging_one_hour_at_capacity_current_adds_100_percent_but_is_clamped(self):
        self.fail("TODO: charge 50 A for 3600 s; expect SOC clamped to 100")

    def test_discharge_never_goes_below_zero(self):
        self.fail("TODO")

    def test_negative_dt_is_rejected(self):
        self.fail("TODO: use assertRaises")

class TestCanMessageCodec(unittest.TestCase):
    """Exercise 2: a two-signal message. Write at least three tests (use subTest)."""

    def setUp(self):
        self.msg = CANMessage(0x2F0, "Engine", 4, (
            Signal("rpm", 0, 16, factor=0.25),
            Signal("coolant", 16, 8, offset=-40),
        ))

    def test_encode_known_bytes(self):
        self.fail("TODO: rpm=3000, coolant=90 -> bytes? (work it out by hand first!)")

```

```

def test_unknown_signal_name(self):
    self.fail("TODO: which exception type does encode() raise?")

def test_decode_wrong_length(self):
    self.fail("TODO")

class TestE2E(unittest.TestCase):
    """Exercise 3: end-to-end protection."""

    def test_protect_then_check_is_ok(self):
        self.fail("TODO")

    def test_corrupted_payload_is_detected(self):
        self.fail("TODO: flip a bit, expect E2EStatus.CRC_ERROR")

    def test_counter_wraps_at_16(self):
        self.fail("TODO: counter 16 behaves like counter 0")

if __name__ == "__main__":
    unittest.main()

```

Lab 2 — pytest fundamentals

Duration: 3 h · **Type:** unit testing, data-driven testing · **Code under test:** `sensors.py` , `bms.py` , `can.py`

Why pytest

Less boilerplate (plain `assert`), far better failure output, fixtures with dependency injection, parametrization, a plugin ecosystem (coverage, Hypothesis, xdist, JUnit XML for CI) — and it runs `unittest` suites unchanged, so migration is gradual.

Learning objectives

Use plain asserts and `pytest.approx` ; assert exceptions with `pytest.raises(match=)` ; build fixtures (yield teardown, fixtures using fixtures, scopes); parametrize with ids; use `tmp_path` , `monkeypatch` , `capsys` ; select tests with markers.

Concepts

- **Fixture** = named, injected, reusable arrange step; teardown after `yield` always runs.
- **Scope:** `function` (default, safest) → `module` → `session` (only for immutable/expensive objects).
- **parametrize** : one test body × many data rows. Give rows `ids` so a failure names the case.
- **Markers:** `smoke` , `xfail(strict=True)` , `skipif` — `--strict-markers` (enabled here) rejects typos.

Run it

```
pytest labs/lab02_pytest -v
pytest labs/lab02_pytest -v -m smoke
pytest labs/lab02_pytest -k "bms and not undervolt" -v
pytest labs/lab02_pytest --setup-show -k soc_clamped # see fixture set-up/teardown
```

Guided tour

Sections 1-5 of `test_lab02_pytest.py` : asserts → parametrization → fixtures → built-ins → markers. Note `battery` fixture: its teardown asserts an **invariant** (SOC in 0..100) after every test using it — one fixture, free protection.

Try this: make one parametrized row wrong. Compare pytest's failure output to Lab 1's.

Exercises — `exercise_lab02.py`

1. Collapse two copy-pasted tests into one parametrized test.
2. Write a *factory fixture* (`sensor_factory`) and test the moving-average filter.
3. `parametrize` with `pytest.param(..., id=..., marks=xfail)` for charge current by SOC.
4. `tmp_path` + error cases for a calibration-file reader.

Debrief

- When does a module-scoped fixture become a hidden coupling between tests?
- `xfail(strict=True)` vs `skip` — what does each tell the next engineer?
- Why is `pytest.approx` needed, and how do `rel` and `abs` differ?

Worked example — source

labs/lab02_pytest/test_lab02_pytest.py

```
"""Lab 2 - pytest: plain asserts, fixtures, parametrization, markers, built-ins.

Run: pytest labs/lab02_pytest -v
"""
import pytest

from autotest.bms import BatteryManagementSystem, BMSFault
from autotest.can import CANFrame, CANMessage, Signal
from autotest.sensors import SensorFault, TemperatureSensor, kmh_to_ms

# ---- 1. plain asserts: pytest rewrites them to show values -----
def test_plain_assert():
    assert kmh_to_ms(72) == pytest.approx(20.0)

def test_approx_for_floats():
    assert 0.1 + 0.2 == pytest.approx(0.3) # never compare floats with ==

def test_exceptions_with_context():
    with pytest.raises(ValueError, match="above maximum") as info:
        Signal("v", 0, 8, maximum=10).to_raw(11)
    assert "v" in str(info.value)

# ---- 2. parametrization: one test body, many cases -----
@pytest.mark.parametrize(
    "cells, temp, expected",
    [
        ([3.7, 3.7, 3.7], 25, BMSFault.NONE),
        ([3.7, 4.21, 3.7], 25, BMSFault.OVERVOLTAGE),
        ([3.7, 2.99, 3.7], 25, BMSFault.UNDERVOLTAGE),
        ([3.7, 3.7, 3.7], 61, BMSFault.OVERTEMPERATURE),
        ([3.7, 3.7, 3.7], -21, BMSFault.UNDERTEMPERATURE),
    ],
    ids=["healthy", "overvolt", "undervolt", "overtemp", "undertemp"],
)
def test_bms_fault_detection(cells, temp, expected):
    assert BatteryManagementSystem().evaluate(cells, temp) is expected
```

```

@pytest.mark.parametrize("dlc", range(0, 9))
def test_every_legal_dlc(dlc):
    assert CANFrame(0x100, bytes(dlc)).dlc == dlc

# ---- 3. fixtures: setup/teardown with dependency injection -----
class FakeADC:
    def __init__(self, *values):
        self.values = list(values)

    def read(self):
        return self.values.pop(0) if len(self.values) > 1 else self.values[0]

@pytest.fixture
def adc():
    return FakeADC(2048)

@pytest.fixture
def sensor(adc): # fixtures can use other fixtures
    return TemperatureSensor(adc)

def test_mid_scale_is_about_55_degrees(sensor):
    assert sensor.read() == pytest.approx(55.0, abs=0.05)

@pytest.fixture
def battery():
    bms = BatteryManagementSystem(capacity_ah=50, soc=50)
    yield bms # everything after yield is teardown
    assert 0 <= bms.soc <= 100, "invariant must hold after EVERY test"

def test_soc_clamped(battery):
    battery.update_soc(1000, 3600)
    assert battery.soc == 100

@pytest.fixture(scope="module")
def engine_message():
    """Module scope: built once, shared (only for immutable objects!)."""
    return CANMessage(0x2F0, "Engine", 4, (Signal("rpm", 0, 16, factor=0.25),))

def test_module_scoped_fixture(engine_message):
    assert engine_message.decode(engine_message.encode({"rpm": 1000}))["rpm"] == 1000

# ---- 4. built-in fixtures -----
def test_tmp_path_gives_each_test_a_private_directory(tmp_path):
    log = tmp_path / "trace.asc"
    log.write_text("0.001 1 123 Rx d 3 01 02 03\n")
    assert log.read_text().startswith("0.001")

def test_monkeypatch_environment(monkeypatch):
    import os
    monkeypatch.setenv("VEHICLE_VARIANT", "EV")
    assert os.environ["VEHICLE_VARIANT"] == "EV" # restored automatically afterwards

def test_capsys_captures_output(capsys):
    print("DTC U0121 set")
    assert "U0121" in capsys.readouterr().out

```

```
# ---- 5. markers: select what to run -----
@pytest.mark.smoke
def test_smoke_sensor_rails(sensor, adc):
    adc.values = [0]
    with pytest.raises(SensorFault):
        sensor.read()

@pytest.mark.skipif(not hasattr(pytest, "approx"), reason="needs pytest.approx")
def test_conditional_skip():
    assert True

@pytest.mark.xfail(reason="BUG-114: resolution 0.25 rpm rounds 0.1 rpm to 0", strict=True)
def test_known_bug_is_tracked():
    sig = Signal("rpm", 0, 16, factor=0.25)
    assert sig.to_raw(0.1) == 1
```

Exercise skeleton — source

labs/lab02_pytest/exercise_lab02.py

```
"""Lab 2 exercises. Run: pytest labs/lab02_pytest/exercise_lab02.py -v"""
import pytest

from autotest.sensors import TemperatureSensor, SensorFault, wheel_speed_kmh
from autotest.bms import max_charge_current

class ScriptedADC:
    def __init__(self, *values):
        self.values = list(values)

    def read(self):
        return self.values.pop(0)

# Exercise 1 - parametrize: convert this copy-pasted test into ONE parametrized test
def test_rails_a():
    with pytest.raises(SensorFault):
        TemperatureSensor(ScriptedADC(0)).read()

def test_rails_b():
    with pytest.raises(SensorFault):
        TemperatureSensor(ScriptedADC(4095)).read()

# Exercise 2 - fixture: build a `sensor_factory` fixture that returns a function
# making a TemperatureSensor from a list of ADC counts, then test read_filtered():
# the average of readings 1024, 2048, 3072 after three calls must be the mid-scale
temperature.
def test_filter_averages_three_samples():
    pytest.fail("TODO")

# Exercise 3 - parametrize with ids and pytest.param(..., marks=pytest.mark.xfail):
# cover max_charge_current at soc in (0, 80, 90, 100) for a 25 degC battery.
def test_charge_current_by_soc():
    pytest.fail("TODO")

# Exercise 4 - use tmp_path + monkeypatch: write a function `load_calibration(path)` in
# THIS file that reads "circumference=1.95" from a file, then test it for a good file,
# a missing file (FileNotFoundError) and a malformed line (ValueError).
```

```
def test_calibration_file():  
    pytest.fail("TODO")
```

Lab 3 — Black-box test design

Duration: 2.5 h · **Type:** specification-based testing · **Code under test:** `bms.max_charge_current`, `BatteryManagementSystem.evaluate`

Why it matters

You can never test all inputs (SOC × temperature is continuous). Test-design techniques choose the *few* inputs most likely to expose defects. ISO 26262-6 explicitly recommends **equivalence classes** and **boundary value analysis** for unit tests of safety-related software.

Learning objectives

Derive tests from a specification **without reading the code**: equivalence partitioning, boundary value analysis (BVA), decision tables, invariants/oracle-free checks.

Techniques

Technique	Idea	In this lab
Equivalence partitioning	inputs that the spec treats alike need one representative	temperature classes <code><0</code> , <code>0..<10</code> , <code>10..45</code> , <code>>45</code>
Boundary value analysis	defects cluster at edges → test <i>on</i> and <i>either side</i>	<code>-0.1/0/0.1</code> , <code>9.9/10/10.1</code> , <code>44.9/45/45.1</code>
Decision table	every combination of conditions → action	<code>cold × taper × hot × full</code> (note the <i>cold+taper interaction</i>)
Invariants	properties true for all inputs	<code>0 ≤ I ≤ rated</code> ; non-increasing in SOC

Run it

```
pytest labs/lab03_black_box -v
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current \
    --tests labs/lab03_black_box/test_lab03_black_box.py      # preview of Lab 12: 22/25 = 88
% (3 equivalent mutants)
```

Exercises — `exercise_lab03.py`

BVA for the six fault limits of `evaluate`, a priority decision table for simultaneous faults, and the "spec is silent" case (empty list) — decide, justify, and log the question.

Debrief

- Why test *both* sides of a boundary rather than only the boundary?
- The decision table has a row (cold + taper) that two single-condition tests would never reach. How did you find it?
- Which of your tests would still be valid if the implementation were rewritten completely? (That is the point of black-box.)

Worked example — source

labs/lab03_black_box/test_lab03_black_box.py

```
"""Lab 3 - test-design techniques (black box) on the BMS charge-current rule.

Spec under test: autotest.bms.max_charge_current (read its docstring FIRST - that is
the 'specification'; we design tests from it without looking at the implementation).
"""

import pytest

from autotest.bms import max_charge_current

RATED = 100.0

# ---- Equivalence partitioning: one representative per class -----
# Temperature classes: <0 | 0..<10 | 10..45 | >45          (SOC fixed at a neutral 50 %)
@pytest.mark.requirement("REQ-BMS-001")
@pytest.mark.parametrize(
    "temp, expected",
    [(-20, 0.0), (5, 25.0), (25, 100.0), (60, 0.0)],
    ids=["too-cold", "cold", "normal", "too-hot"],
)
def test_temperature_partitions(temp, expected):
    assert max_charge_current(soc=50, temp_c=temp) == pytest.approx(expected)

# ---- Boundary value analysis: values ON and either side of every boundary -----
@pytest.mark.requirement("REQ-BMS-001")
@pytest.mark.parametrize(
    "temp, expected",
    [
        (-0.1, 0.0), (0.0, 25.0), (0.1, 25.0),      # 0 degC boundary
        (9.9, 25.0), (10.0, 100.0), (10.1, 100.0),  # 10 degC boundary
        (44.9, 100.0), (45.0, 100.0), (45.1, 0.0),  # 45 degC boundary
    ],
)
def test_temperature_boundaries(temp, expected):
    assert max_charge_current(soc=50, temp_c=temp) == pytest.approx(expected)

@pytest.mark.requirement("REQ-BMS-002")
@pytest.mark.parametrize(
    "soc, expected",
    [(0, 100.0), (79.9, 100.0), (80.0, 100.0), (80.1, 99.5), (90, 50.0), (99, 5.0), (100,
0.0)],
)
def test_soc_taper_boundaries(soc, expected):
    assert max_charge_current(soc=soc, temp_c=25) == pytest.approx(expected)

@pytest.mark.parametrize("soc", [-0.1, 100.1])
def test_invalid_soc_is_rejected(soc):
```

```

    with pytest.raises(ValueError):
        max_charge_current(soc=soc, temp_c=25)

# ---- Decision table: every combination of conditions -> expected action -----
#   cold | hot   | soc>80 | soc>=100 | result
#   -----+-----+-----+-----+-----
#   no    | no    | no     | no      | full current
#   cold  |      |        |         | 25 % current
#   no    | no    | yes    | no      | tapered
#   cold  | no    | yes    | no      | 25 % AND tapered (rules combine!)
#   any   | hot   | any    | any     | 0
DECISION_TABLE = [
    # soc, temp, expected
    (50, 25, 100.0), # normal
    (50, 5, 25.0),   # cold only
    (90, 25, 50.0),  # taper only
    (90, 5, 12.5),   # cold + taper (interaction - the classic missed case)
    (50, 50, 0.0),   # hot
    (90, 50, 0.0),   # hot beats taper
    (100, 25, 0.0),  # full
]

@pytest.mark.parametrize("soc, temp, expected", DECISION_TABLE)
def test_decision_table(soc, temp, expected):
    assert max_charge_current(soc, temp) == pytest.approx(expected)

# ---- Invariants / oracle-free checks -----
def test_current_never_negative_or_above Rated_over_whole_grid():
    for soc in range(0, 101, 5):
        for temp in range(-30, 71, 5):
            i = max_charge_current(soc, temp)
            assert 0.0 <= i <= RATED, (soc, temp, i)

def test_current_is_monotonic_non_increasing_in_soc():
    values = [max_charge_current(s / 2, 25) for s in range(0, 201)]
    assert values == sorted(values, reverse=True)

```

Exercise skeleton — source

labs/lab03_black_box/exercise_lab03.py

```

"""Lab 3 exercises. Run: pytest labs/lab03_black_box/exercise_lab03.py -v

Specification - BatteryManagementSystem.evaluate(cell_voltages, temp_c):
    returns OVERVOLTAGE if any cell > 4.20 V, UNDERVOLTAGE if any cell < 3.00 V,
    OVERTEMPERATURE if temp > 60, UNDERTEMPERATURE if temp < -20, else NONE.
    (Checks are made in that order of priority.)
"""
import pytest

from autotest.bms import BatteryManagementSystem, BMSFault

# Exercise 1 - Boundary values: list the 6 boundary cases (on / just beyond for each limit)
# and put them in a parametrized test. Use 4.20 +/- 0.01 etc.
def test_boundaries():
    pytest.fail("TODO")

# Exercise 2 - Decision table: write the table of the 4 interesting COMBINATIONS
# (over+under voltage, over-voltage+over-temp, ...) as a comment, then test the priority

```

```
rule.  
    def test_priority_between_faults():  
        pytest.fail("TODO")  
  
    # Exercise 3 - Invalid input: an empty list of cells. Decide from the spec what must happen  
    # and justify in a comment (is the spec silent? then raise a question to the 'product  
owner').  
    def test_empty_cell_list():  
        pytest.fail("TODO")
```

Lab 4 — Test doubles: isolating the unit

Duration: 2.5 h · **Type:** isolation techniques · **Code under test:** `TemperatureSensor`, `InstrumentCluster`, `WheelSpeedECU`

Why it matters

Real ECU software talks to ADCs, buses, NVM and other ECUs. On a CI server none of those exist, and some situations (short-to-ground, 100 ms bus silence) cannot be produced safely or repeatably on hardware. Doubles make the world scriptable.

The five doubles (Meszaros)

Double	Role	Example here
Dummy	placeholder	a listener on an id that never fires
Stub	canned answers	<code>StubADC(0)</code>
Fake	simplified real implementation	<code>SineWaveADC</code> , <code>FakeClock</code> , <code>VirtualCANBus</code>
Spy	records usage	<code>SpyADC.calls</code>
Mock	verifies expected interactions	<code>Mock(spec=DiagnosticManager)</code>

Key skills

- `mock.Mock(spec=Class)` — typos in method names raise instead of silently passing.
- `side_effect=[...]` for sequences and exceptions; `call_count`, `assert_called_with`.
- **Patch where the name is looked up** (`autotest.ecu.e2e_protect`, not `autotest.can.e2e_protect`).
- **Inject time:** `FakeClock.advance(0.5)` replaces `time.sleep(0.5)` — fast and deterministic.

Run it

```
pytest labs/lab04_test_doubles -v
```

Exercises — `exercise_lab04.py`

Spy on the alive counter · simulate an `OSError` from the ADC · timeout boundary at 99/101 ms with a fake clock · write 3 sentences on when a mock-based test is *bad*.

Debrief

- The last worked test prefers the **real** `DiagnosticManager` over a mock. Why? What does the mock version lock in?
- What would break if `ecu.py` did `from .can import e2e_protect` and you patched `autotest.can.e2e_protect`?
- Which doubles would you need to test a *real* `BenchADC` driver?

Worked example — source

labs/lab04_test_doubles/test_lab04_test_doubles.py

```
"""Lab 4 - test doubles: dummy, stub, fake, spy and mock.

The SUT depends on hardware (an ADC) and on a diagnostic manager. We replace those
collaborators so the test is fast, deterministic, and can force rare situations.
"""

from unittest import mock

import pytest

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import DTC_COMM_LOST, InstrumentCluster, WheelSpeedECU
from autotest.sensors import SensorFault, TemperatureSensor

# ---- STUB: returns canned answers -----
class StubADC:
    def __init__(self, counts):
        self.counts = counts

    def read(self):
        return self.counts

def test_stub_forces_a_wiring_fault_that_real_hardware_rarely_produces():
    with pytest.raises(SensorFault, match="short to ground"):
        TemperatureSensor(StubADC(0)).read()

# ---- FAKE: a working but simplified implementation -----
class SineWaveADC:
    """Plays back a sequence like a data logger would."""

    def __init__(self, samples):
        self._it = iter(samples)

    def read(self):
        return next(self._it)

def test_fake_playback_feeds_the_moving_average():
    sensor = TemperatureSensor(SineWaveADC([1000, 2000, 3000, 2000]), filter_len=2)
    readings = [sensor.read_filtered() for _ in range(4)]
    expected_second = (TemperatureSensor.T_MIN + 190 * 1000 / 4095 + TemperatureSensor.T_MIN
+ 190 * 2000 / 4095) / 2
    assert readings[1] == pytest.approx(expected_second)
    assert readings[0] < readings[2] # trend follows the input
```

```

# ---- SPY: real object that records how it was used -----
class SpyADC:
    def __init__(self, counts):
        self.counts, self.calls = counts, 0

    def read(self):
        self.calls += 1
        return self.counts

def test_spy_counts_hardware_accesses():
    spy = SpyADC(2000)
    sensor = TemperatureSensor(spy, filter_len=3)
    for _ in range(3):
        sensor.read_filtered()
    assert spy.calls == 3 # exactly one ADC conversion per call: no hidden re-reads

# ---- MOCK (unittest.mock): verify INTERACTIONS -----
def test_mock_verifies_the_cluster_reports_to_diagnostics(clock):
    bus = VirtualCANBus()
    diag = mock.Mock(spec=DiagnosticManager) # spec= makes typos in method names fail
    cluster = InstrumentCluster(bus, clock, diag)

    clock.advance(0.5) # no frames for 500 ms
    assert cluster.displayed_speed() == "--"

    diag.report.assert_called_with(DTC_COMM_LOST, failed=True)

def test_mock_side_effect_simulates_intermittent_failure():
    adc = mock.Mock()
    adc.read.side_effect = [2048, 2048, 0, 2048] # third read: fault
    sensor = TemperatureSensor(adc)
    assert sensor.read() == pytest.approx(55.0, abs=0.05)
    sensor.read()
    with pytest.raises(SensorFault):
        sensor.read()
    assert adc.read.call_count == 3

# ---- patching: replace a name where it is LOOKED UP -----
def test_patch_replaces_collaborator_for_the_duration_of_the_test():
    with mock.patch("autotest.ecu.e2e_protect", return_value=b"\x00\x00\x00\x00\x00") as
fake:
        bus = VirtualCANBus()
        WheelSpeedECU(bus).send_speed(50)
        fake.assert_called_once()
        assert bus.log[0].data == b"\x00\x00\x00\x00\x00"

def test_monkeypatch_equivalent_in_pytest(monkeypatch):
    monkeypatch.setattr("autotest.ecu.e2e_protect", lambda payload, counter: b"\xAA" +
payload)
        bus = VirtualCANBus()
        WheelSpeedECU(bus).send_speed(10)
        assert bus.log[0].data[0] == 0xAA

# ---- DUMMY: fills a parameter that is never used -----
def test_dummy_listener_is_never_called_for_other_ids():
    bus = VirtualCANBus()
    listener = mock.Mock()
    bus.subscribe(listener, ids={0x100})
    bus.send(CANFrame(0x200, b"\x01"))
    listener.assert_not_called()

```

```
# ---- Anti-pattern warning -----
def test_do_not_over_mock_real_collaborator_is_cheaper_here(clock):
    """The real DiagnosticManager is pure and fast, so using it beats a mock:
    we assert on OUTCOMES (state), not on call choreography."""
    diag = DiagnosticManager(fail_threshold=2)
    cluster = InstrumentCluster(VirtualCANBus(), clock, diag)
    clock.advance(1)
    cluster.displayed_speed()
    cluster.displayed_speed()
    assert diag.active_codes() == [DTC_COMM_LOST]
```

Exercise skeleton — source

labs/lab04_test_doubles/exercise_lab04.py

```
"""Lab 4 exercises. Run: pytest labs/lab04_test_doubles/exercise_lab04.py -v"""
import pytest
from unittest import mock

from autotest.bus import VirtualCANBus
from autotest.ecu import InstrumentCluster, WheelSpeedECU
from autotest.sensors import TemperatureSensor

# Exercise 1 - SPY: write SpyListener (records frames). Send 3 speeds from WheelSpeedECU and
# assert the alive counter in data[1] goes 0, 1, 2.
def test_alive_counter_increments():
    pytest.fail("TODO")

# Exercise 2 - MOCK: use mock.Mock(spec=...) for an ADC whose read() raises OSError once.
# What does TemperatureSensor do? Document the behaviour with pytest.raises.
def test_adc_hardware_error_propagates():
    pytest.fail("TODO")

# Exercise 3 - FAKE: write FakeClock-driven test: speed frame at t=0, ask at t=0.09 s (shows
number)
# and at t=0.11 s (shows "--"). Why is time.sleep() the wrong tool here?
def test_cluster_timeout_boundary(clock):
    pytest.fail("TODO")

# Exercise 4 - Reflection (write 3 sentences in a comment): when is a mock-based test BAD?
```

Lab 5 — Integration testing

Duration: 3 h · **Type:** integration / interface / contract testing · **Code under test:** `WheelSpeedECU` → `VirtualCANBus` → `InstrumentCluster`

Why it matters

Most field failures sit *between* components: one team scales by 0.01, another by 0.1; the alive counter wraps and the receiver rejects it; a CRC covers the wrong bytes. Unit tests of each side pass; the system is wrong.

Learning objectives

Test an interface end-to-end through a virtual bus; inject faults (corruption, loss, noise) with bus hooks; verify E2E protection and timeouts; write a **contract test** against the exact bytes on the wire; recognise smoke tests.

Strategy: integration order

Bottom-up (codec → ECU → bus → cluster) was used to build the SUT; *top-down* with stubs is the alternative. Choose by risk — integrate the riskiest interface first.

Run it

```
pytest labs/lab05_integration -v
pytest labs/lab05_integration -m smoke # a single, fastest end-to-end check
```

Guided tour

- `test_speed_travels_from_sensor_ecu_to_display` — the **smoke** test.
- `test_frame_on_the_wire_matches_the_interface_specification` — byte-level contract.
- `test_cluster_blanks_after_100ms...` — timing via `FakeClock`, both sides of 100 ms.
- `test_corrupted_frame...`, `test_lost_frame...` — `bus.add_fault(hook)`; a hook returns a modified frame or `None`.

Real finding: writing the Lab 9 replay later exposed that one corrupted frame cost two frames (BUG-130). Integration tests are where such cross-component sequencing defects hide.

Exercises — `exercise_lab05.py`

Add a third node (data logger) · seeded random bit-flips on 1 in 5 frames · a contract test that fails if the scale factor changes.

Debrief

- What does a contract test protect that a round-trip test cannot?
- Why must the fault-injection RNG be seeded?
- Which failures can only be found at integration level? Which would you rather find at unit level?

Worked example — source

labs/lab05_integration/test_lab05_integration.py

```
"""Lab 5 - integration testing: WheelSpeedECU -> VirtualCANBus -> InstrumentCluster.

Unit tests prove each part works alone; these prove the parts agree on the INTERFACE:
byte layout, scaling, E2E protection, alive counter and timing.
"""

import pytest

from autotest.can import CANFrame
from autotest.ecu import (
    DTC_COMM_LOST, DTC_E2E_FAIL, WHEEL_SPEED_MSG, InstrumentCluster, WheelSpeedECU,
)

pytestmark = pytest.mark.integration

@pytest.fixture
def ecu(bus):
    return WheelSpeedECU(bus)

@pytest.fixture
def cluster(bus, clock):
    return InstrumentCluster(bus, clock)

@pytest.mark.smoke
def test_speed_travels_from_sensor_ecu_to_display(ecu, cluster):
    ecu.send_speed(87.4)
    assert cluster.displayed_speed() == "87"

@pytest.mark.parametrize("speed", [0, 0.01, 50, 130.55, 299.99])
def test_interface_scaling_agrees_end_to_end(ecu, cluster, speed):
    ecu.send_speed(speed)
    assert cluster.displayed_speed() == f"{speed:.0f}"

def test_frame_on_the_wire_matches_the_interface_specification(ecu, bus):
    ecu.send_speed(100.0)
    frame = bus.log[0]
    assert frame.arbitration_id == 0x1A0
    assert frame.dlc == 5 # crc + counter + 3 payload bytes
    assert frame.data[2:4] == (10000).to_bytes(2, "little") # 100.00 km/h, factor 0.01
```

```

@pytest.mark.requirement("REQ-CLU-001")
def test_cluster_blanks_after_100ms_without_frames(ecu, cluster, clock):
    ecu.send_speed(60)
    clock.advance(0.099)
    assert cluster.displayed_speed() == "60"
    clock.advance(0.002) # total 101 ms
    assert cluster.displayed_speed() == "--"

@pytest.mark.requirement("REQ-CLU-001")
def test_cluster_recovers_when_frames_return(ecu, cluster, clock):
    ecu.send_speed(60)
    clock.advance(1)
    assert cluster.displayed_speed() == "--"
    ecu.send_speed(70)
    assert cluster.displayed_speed() == "70"

@pytest.mark.requirement("REQ-CAN-003")
def test_corrupted_frame_is_rejected_and_last_good_value_kept(ecu, cluster, bus):
    ecu.send_speed(50)
    bus.add_fault(lambda f: CANFrame(f.arbitration_id, bytes([f.data[0] ^ 0xFF]) +
f.data[1:]))
    ecu.send_speed(120) # CRC byte corrupted in flight
    assert cluster.displayed_speed() == "50"
    assert cluster.diag.stored_codes() == [] # one failure is only debounced...
    ecu.send_speed(121)
    assert cluster.diag.stored_codes() == [DTC_E2E_FAIL] # ...two confirm the DTC

@pytest.mark.requirement("REQ-CAN-003")
def test_lost_frame_is_detected_by_alive_counter_and_cluster_resyncs(ecu, cluster, bus):
    ecu.send_speed(10)
    bus.add_fault(lambda f: None) # bus drops one frame
    ecu.send_speed(20)
    bus.clear_faults()
    ecu.send_speed(30) # counter jumped by 2 -> rejected once
    assert cluster.displayed_speed() == "10"
    ecu.send_speed(40) # resynchronised -> accepted
    assert cluster.displayed_speed() == "40"

def test_cluster_ignores_other_message_ids(ecu, cluster, bus):
    ecu.send_speed(33)
    bus.send(CANFrame(0x7FF, b"\xFF" * 8)) # unrelated traffic
    assert cluster.displayed_speed() == "33"

def test_comm_lost_dtc_is_confirmed_by_debounce(cluster, clock):
    clock.advance(1)
    cluster.displayed_speed()
    assert cluster.diag.active_codes() == [] # first miss: only counted
    cluster.displayed_speed()
    assert cluster.diag.active_codes() == [DTC_COMM_LOST]

def test_bus_log_contains_every_frame_even_when_dropped(ecu, bus):
    bus.add_fault(lambda f: None)
    assert ecu.send_speed(10) is False
    assert len(bus.log) == 1

```

Exercise skeleton — source

labs/lab05_integration/exercise_lab05.py

```

"""Lab 5 exercises. Run: pytest labs/lab05_integration/exercise_lab05.py -v"""
import pytest

from autotest.ecu import InstrumentCluster, WheelSpeedECU

# Exercise 1 - Top-down vs bottom-up: add a THIRD node (write a tiny `DataLogger` class in
this
# file that subscribes to the bus and stores decoded speeds). Integrate it and assert it sees
# exactly the frames the cluster accepted.
def test_logger_sees_what_cluster_sees(bus, clock):
    pytest.fail("TODO")

# Exercise 2 - Fault injection on the bus: a hook that flips ONE random-but-seeded bit
# in 1 of every 5 frames (use random.Random(42)). Send 100 frames. Assert the cluster
# never displays a value that was not sent.
def test_cluster_never_displays_invented_speed(bus, clock):
    pytest.fail("TODO")

# Exercise 3 - Interface contract: write a test that FAILS if someone changes the scale
factor
# of WHEEL_SPEED_MSG without updating the receiver. (Hint: compare against raw bytes
# captured on the wire - a 'contract test'.)
def test_wheel_speed_contract():
    pytest.fail("TODO")

```

Lab 6 — State-transition and system-level testing

Duration: 3 h · **Type:** state-based testing, functional/system testing · **Code under test:**

`CruiseController`

Why it matters

Mode logic (OFF / STANDBY / ACTIVE / OVERRIDE) is a classic place for hazardous bugs: cruise that stays active after braking, or re-engages unexpectedly. Behaviour depends on *history*, so single-input tests are not enough.

Learning objectives

Model behaviour as states × events; cover **all states, all valid transitions, all invalid events**; use an exhaustive short-sequence test with invariants; test the control law (saturation, anti-windup) at the same time.

Coverage levels

Level	Meaning	Here
All-states	every state visited	covered by transition tests
0-switch (all transitions)	every valid arrow taken	<code>test_off_to_standby ...</code> <code>test_accelerator_overrides...</code>
Invalid-event coverage	every event in every state	exhaustive 3-event sequences (343 cases) + exercise 1's 28-row table
n-switch	sequences of n transitions	<code>itertools.product(EVENTS, repeat=3)</code>

Run it

```
pytest labs/lab06_state_machine -v -k "not sequences"
pytest labs/lab06_state_machine -q          # includes 343 generated sequence tests
```

Guided tour

The state diagram is in the module docstring — **draw it before reading the tests**. Observe: the sequence test asserts *invariants* (`ACTIVE ⇔ target is set` , throttle is 0 outside ACTIVE) instead of exact outputs, so it can run over every sequence with no oracle for each one.

`test_integral_does_not_wind_up...` is a deliberate **white-box** check.

Exercises — `exercise_lab06.py`

28-row transition table · "overtaking" scenario in Given/When/Then with the plant model · a design question (`resume` above max).

Debrief

- Which invalid transitions are *safety relevant* here? Would you want a DTC on them?
- What is the difference between asserting `state` and asserting the externally visible *behaviour* (throttle)?
- How many states/events before exhaustive sequence testing becomes impractical — and what do you do then?

Worked example — source

`labs/lab06_state_machine/test_lab06_state_machine.py`

```
"""Lab 6 - state-transition and system-level testing of the cruise controller.

Technique: draw the state machine, then cover (a) every state, (b) every VALID transition,
(c) every INVALID event in every state ("0-switch" and invalid-transition coverage).

    power_on          set(v)          brake/cancel
OFF -----> STANDBY -----> ACTIVE -----> STANDBY
 ^    <--- power_off (any state) --- | ^
                                accel v | release
                                OVERRIDE

"""
import itertools

import pytest

from autotest.cruise import CruiseController, CruiseState as S

pytestmark = pytest.mark.system

@pytest.fixture
def cc():
    return CruiseController()

@pytest.fixture
def active(cc):
    cc.power_on()
    assert cc.set(100)
    return cc

# ---- valid transitions -----
def test_initial_state_is_off(cc):
    assert cc.state is S.OFF and cc.target is None

def test_off_to_standby(cc):
    cc.power_on()
```

```

    assert cc.state is S.STANDBY

@pytest.mark.requirement("REQ-CC-001")
def test_standby_to_active_with_set(cc):
    cc.power_on()
    assert cc.set(100) is True
    assert (cc.state, cc.target) == (S.ACTIVE, 100)

@pytest.mark.requirement("REQ-CC-002")
def test_brake_cancels_and_remembers_target(active):
    active.brake()
    assert active.state is S.STANDBY and active.target is None
    assert active.saved_target == 100

def test_resume_restores_saved_target(active):
    active.cancel()
    assert active.resume(current_speed_kmh=90) is True
    assert (active.state, active.target) == (S.ACTIVE, 100)

def test_accelerator_overrides_then_returns_to_active(active):
    active.accelerator(True)
    assert active.state is S.OVERRIDE
    active.accelerator(False)
    assert active.state is S.ACTIVE

def test_set_while_active_changes_target(active):
    assert active.set(120)
    assert active.target == 120

@pytest.mark.parametrize("state_setup", ["off", "standby", "active", "override"])
def test_power_off_from_every_state_clears_everything(cc, state_setup):
    if state_setup != "off":
        cc.power_on()
    if state_setup in ("active", "override"):
        cc.set(100)
    if state_setup == "override":
        cc.accelerator(True)
    cc.power_off()
    assert (cc.state, cc.target, cc.saved_target) == (S.OFF, None, None)

# ---- invalid events (negative transitions) -----
def test_set_is_refused_when_off(cc):
    assert cc.set(100) is False and cc.state is S.OFF

@pytest.mark.requirement("REQ-CC-001")
@pytest.mark.parametrize("speed, accepted", [
    (29.9, False), (30.0, True), (30.1, True), (179.9, True), (180.0, True), (180.1, False),
])
def test_set_speed_window_boundaries(cc, speed, accepted):
    cc.power_on()
    assert cc.set(speed) is accepted
    assert cc.state is (S.ACTIVE if accepted else S.STANDBY)

def test_resume_without_history_is_refused(cc):
    cc.power_on()
    assert cc.resume(100) is False

def test_resume_below_minimum_speed_is_refused(active):

```

```

    active.cancel()
    assert active.resume(20) is False and active.state is S.STANDBY

def test_accelerator_is_ignored_in_standby(cc):
    cc.power_on()
    cc.accelerator(True)
    assert cc.state is S.STANDBY

def test_cancel_in_off_is_a_noop(cc):
    cc.cancel()
    assert cc.state is S.OFF

# ---- exhaustive: every event in every state never crashes and keeps invariants -----
EVENTS = {
    "power_on": lambda c: c.power_on(),
    "power_off": lambda c: c.power_off(),
    "set": lambda c: c.set(100),
    "resume": lambda c: c.resume(100),
    "cancel": lambda c: c.cancel(),
    "accel_on": lambda c: c.accelerator(True),
    "accel_off": lambda c: c.accelerator(False),
}

@pytest.mark.parametrize("sequence", list(itertools.product(EVENTS, repeat=3)), ids="-".join)
def test_all_3_event_sequences_keep_invariants(sequence):
    cc = CruiseController()
    for name in sequence:
        EVENTS[name](cc)
        assert (cc.state is S.ACTIVE) == (cc.target is not None) or cc.state is S.OVERRIDE
        assert cc.control(100, 0.1) >= 0
        if cc.state is not S.ACTIVE:
            assert cc.control(50, 0.1) == 0.0

# ---- control law: system behaviour with a simple vehicle model -----
@pytest.mark.requirement("REQ-CC-003")
def test_throttle_is_zero_unless_active(cc):
    assert cc.control(50, 0.1) == 0.0
    cc.power_on()
    assert cc.control(50, 0.1) == 0.0

def test_throttle_is_saturated_between_0_and_1(active):
    assert active.control(30, 0.1) == 1.0      # far below target -> full throttle
    assert active.control(150, 0.1) == 0.0    # above target -> no throttle

def test_integral_does_not_wind_up_during_saturation(active):
    for _ in range(1000):
        active.control(30, 0.1)
    assert active._integral == 0.0            # white-box check of the anti-windup rule

```

Exercise skeleton — source

labs/lab06_state_machine/exercise_lab06.py

```

"""Lab 6 exercises. Run: pytest labs/lab06_state_machine/exercise_lab06.py -v"""
import pytest

from autotest.cruise import CruiseController, CruiseState

```

```

# Exercise 1 - Draw a transition table (state x event -> next state) in a comment for ALL
# 4 states x 7 events, then implement it as ONE parametrized test with 28 rows.
def test_full_transition_table():
    pytest.fail("TODO")

# Exercise 2 - A scenario ("system test") in Given/When/Then style:
# Given the driver cruises at 100, When they overtake (accelerator pressed 5 s, released),
# Then cruise returns to ACTIVE with the SAME target and no throttle spike (< 0.5).
def test_overtaking_scenario():
    pytest.fail("TODO")

# Exercise 3 - Find a design question: what should resume() do when the car is above 180?
# Write the test for the behaviour you CHOOSE, and file the ambiguity as a comment.
def test_resume_above_max():
    pytest.fail("TODO")

```

Lab 7 — Property-based testing with Hypothesis

Duration: 3 h · **Type:** generative testing · **Code under test:** signal codec, E2E, `max_charge_current`

Why it matters

Example-based tests check the cases *you* thought of. Property-based tests state a rule that must hold for **all** inputs; Hypothesis generates hundreds of cases, then **shrinks** a failure to the smallest counter-example. It routinely finds the `0x7FF` vs `0x800` kind of bug nobody listed.

Learning objectives

Find properties (round-trip, invariant, idempotence, oracle/model, "never crashes"); write strategies (`st.floats`, `st.builds`, `st.data`); control the search (`@settings`, `@example`, `assume`); read a shrunk failure.

Property catalogue (automotive flavour)

Pattern	Example
Round trip	<code>decode(encode(x)) ≈ x</code> within one resolution step
Detection guarantee	CRC detects <i>any</i> single-bit flip
Safety invariant	never charge below 0 °C or above 45 °C, whatever SOC
Monotonicity	charge current never increases with SOC
Reference model	a 10-line reference DTC debouncer vs the real manager (exercise 2)
Crash-resistance	<code>CANFrame(...)</code> either builds or raises <code>ValueError</code> — nothing else

Run it

```
pytest labs/lab07_property_based -v
pytest labs/lab07_property_based --hypothesis-show-statistics
pytest labs/lab07_property_based --hypothesis-seed=0 # reproduce a CI failure
```

Guided tour

- `test_wrong_property_resolution_is_exact` is an `xfail` — remove the marker and read how Hypothesis shrinks a failure.

- The `assume()` anti-pattern: our first draft filtered two independent floats and Hypothesis raised `FailedHealthCheck(filter_too_much)`. The fix — generate the *delta* directly — is the rule: **generate valid data; don't filter invalid data.**

Exercises — `exercise_lab07.py`

Properties of `slip_ratio` · DTC debouncer vs a reference model · a `st.builds(CANFrame, ...)` strategy.

Debrief

- Why is a round-trip property weaker than an example with a known byte string? What does each catch?
- Hypothesis stores failures in `.hypothesis/`. How should CI treat that directory?
- What makes a *good* property — and why is "output equals a re-implementation of the code" a bad one?

Worked example — source

labs/lab07_property_based/test_lab07_property_based.py

```

"""Lab 7 - property-based testing with Hypothesis.

Instead of hand-picking inputs, state a PROPERTY that must hold for ALL inputs and let
Hypothesis search (and shrink) counter-examples.
"""

import pytest
from hypothesis import import assume, example, given, settings, strategies as st

from autotest.bms import max_charge_current
from autotest.can import CANFrame, CANMessage, E2EStatus, Signal, crc8, e2e_check,
e2e_protect
from autotest.sensors import kmh_to_ms, ms_to_kmh

payloads = st.binary(min_size=0, max_size=6)

# --- property 1: round trip (encode then decode == identity) -----
@pytest.mark.requirement("REQ-CAN-002")
@given(st.floats(min_value=0, max_value=300, allow_nan=False))
def test_wheel_speed_round_trip_within_one_resolution_step(speed):
    sig = Signal("speed", 0, 16, factor=0.01, minimum=0, maximum=300)
    assert sig.to_physical(sig.to_raw(speed)) == pytest.approx(speed, abs=0.005 + 1e-9)

@given(st.integers(0, 65535), st.integers(-40, 215))
def test_message_round_trip_for_any_raw_values(rpm_raw, temp_raw):
    msg = CANMessage(1, "m", 4, (Signal("rpm", 0, 16, 0.25), Signal("t", 16, 8, offset=-40)))
    values = {"rpm": rpm_raw * 0.25, "t": float(temp_raw)}
    assert msg.decode(msg.encode(values)) == pytest.approx(values)

@given(st.floats(min_value=-1e4, max_value=1e4))
def test_unit_conversion_round_trip(kmh):
    assert ms_to_kmh(kmh_to_ms(kmh)) == pytest.approx(kmh)

```

```

# ---- property 2: end-to-end protection detects every corruption we can generate -----
@pytest.mark.requirement("REQ-CAN-003")
@given(payloads, st.integers(0, 15))
def test_e2e_accepts_what_it_protects(payload, counter):
    status, out = e2e_check(e2e_protect(payload, counter), counter)
    assert status is E2EStatus.OK and out == payload

@pytest.mark.requirement("REQ-CAN-003")
@given(payloads, st.integers(0, 15), st.data())
def test_e2e_detects_any_single_bit_flip(payload, counter, data):
    protected = bytearray(e2e_protect(payload, counter))
    bit = data.draw(st.integers(0, len(protected) * 8 - 1))
    protected[bit // 8] ^= 1 << (bit % 8)
    status, _ = e2e_check(bytes(protected))
    assert status is not E2EStatus.OK

@given(st.binary(max_size=8))
def test_crc_is_always_one_byte_and_deterministic(data):
    assert 0 <= crc8(data) <= 255 and crc8(data) == crc8(data)

# ---- property 3: safety invariants of the BMS -----
@pytest.mark.requirement("REQ-BMS-001")
@given(st.floats(0, 100), st.floats(-60, 120))
def test_charge_current_is_always_within_physical_limits(soc, temp):
    i = max_charge_current(soc, temp)
    assert 0.0 <= i <= 100.0
    if temp < 0 or temp > 45:
        assert i == 0.0 # safety property: never charge outside the temperature window

@given(st.floats(0, 100), st.floats(0, 100), st.floats(10, 45))
def test_charge_current_never_increases_with_soc(a, b, temp):
    low, high = sorted((a, b))
    assert max_charge_current(high, temp) <= max_charge_current(low, temp) + 1e-9

# ---- property 4: validation never crashes in unexpected ways -----
@given(st.integers(-10_000, 0x30000000), st.booleans(), st.binary(max_size=12))
def test_canframe_either_builds_or_raises_valueerror(arb_id, ext, data):
    try:
        frame = CANFrame(arb_id, data, ext)
    except ValueError:
        return
    assert frame.dlc == len(data) <= 8 and 0 <= frame.arbitration_id <= (0x1FFFFFFF if ext
else 0x7FF)

# ---- steering Hypothesis: @example, settings, assume() (and why NOT to over-filter) -----
-
# Anti-pattern: `assume(abs(a - b) < 0.01)` on two independent floats rejects ~100 % of
inputs
# and Hypothesis aborts with FailedHealthCheck(filter_too_much). Generate the SMALL DELTA
directly.
@settings(max_examples=200, deadline=None)
@given(st.floats(0, 99.99), st.floats(0, 0.01))
@example(80.0, 0.0) # always re-test this edge case, in addition to random ones
def test_taper_is_continuous_at_80_percent(soc, delta):
    assume(soc + delta <= 100) # cheap, rarely-triggered filter: fine
    assert abs(max_charge_current(soc, 25) - max_charge_current(soc + delta, 25)) < 1.0

# ---- the point of shrinking: a deliberately wrong property, kept as a documented xfail ----
-
@pytest.mark.xfail(strict=True, reason="demo: Hypothesis shrinks to the smallest counter-
example")

```

```

@given(st.floats(min_value=0, max_value=300))
def test_wrong_property_resolution_is_exact(speed):
    sig = Signal("speed", 0, 16, factor=0.01)
    assert sig.to_physical(sig.to_raw(speed)) == speed # false: quantisation error

```

Exercise skeleton — source

labs/lab07_property_based/exercise_lab07.py

```

"""Lab 7 exercises. Run: pytest labs/lab07_property_based/exercise_lab07.py -v"""
import pytest
from hypothesis import given, strategies as st

from autotest.abs import slip_ratio
from autotest.diagnostics import DiagnosticManager

# Exercise 1 - properties of slip_ratio: (a) always within [0, 1], (b) 0 when wheel ==
vehicle,
# (c) 1 when the wheel is stopped and the car is moving > 0.5 m/s. Write one @given test
each.
def test_slip_properties():
    pytest.fail("TODO")

# Exercise 2 - Stateful thinking: generate lists of booleans (st.lists(st.booleans())) as DTC
# pass/fail histories. Property: a code is active iff the LAST `fail_threshold` results are
all
# failures OR (it was active before and fewer than `heal_threshold` passes followed).
# Write a tiny reference model in the test and compare it with DiagnosticManager.
def test_dtc_matches_reference_model():
    pytest.fail("TODO")

# Exercise 3 - Custom strategy: build st.builds(CANFrame, ...) with valid ids, then assert
# that e2e_protect(frame.data[:6], 3) is always a valid input for e2e_check.
def test_custom_strategy():
    pytest.fail("TODO")

```

Lab 8 — Robustness: negative testing, fault injection and fuzzing

Duration: 3 h · **Type:** robustness / fault-injection / fuzz · **Code under test:** sensor, bus, cluster, `DiagnosticManager.handle_request`, BMS

Why it matters

Functional correctness answers "does it work?". Robustness answers "**does it fail safely?**" — the question behind ISO 26262 safety mechanisms, ISO 21434 cybersecurity and the UDS diagnostic port that anybody with an OBD dongle can reach.

Learning objectives

Inject sensor faults with a decorator (stuck, short, open); inject bus faults with seeded randomness; fuzz an input parser and assert **universal** properties (never raises, always answers with a valid response); recognise hostile numerics (NaN, $\pm\text{inf}$); verify recovery and DTC history.

Techniques

Technique	Where
Fault-injection decorator (<code>FaultyADC</code>)	wraps any ADC
Bus fault hook with seeded RNG	<code>test_cluster_never_displays_an_invented_speed...</code>
Fuzzing: Hypothesis and a plain seeded loop	UDS handler
Documented gaps	stuck sensor is <i>not</i> detected today — the test says so
Strict <code>xfail</code> for known defects	BUG-201 / BUG-202 below

Two real defects found by this lab

- **BUG-201** `max_charge_current(50, nan)` returns **100 A** — NaN fails every comparison, so no limit applies. *Safety-relevant*.
- **BUG-202** `evaluate([3.7, nan, 3.7], 25)` reports `NONE` (healthy).

They are pinned with `xfail(strict=True)` : the suite stays green now and goes **red when someone fixes the bug**, forcing the marker to be removed. `solutions/bms_nan_fix.patch` is the verified fix.

Run it

```
pytest labs/lab08_robustness -v -rx
```

Exercises — exercise_lab08.py

Fix BUG-201/202 and clean up the markers · TDD a "stuck sensor" plausibility check · model a 5-frame burst loss and show recovery.

Debrief

- What does "fail-safe" mean for a charge-current limit when the temperature input is invalid?
- Why print/seed the RNG in fuzz tests?
- Fuzzing found nothing in the UDS handler. Does that prove it is secure? What *would* you need?

Worked example — source

labs/lab08_robustness/test_lab08_robustness.py

```
"""Lab 8 - robustness: negative tests, fault injection and fuzzing.

Question changes from "does it work?" to "does it fail SAFELY?".
"""
import math
import random

import pytest
from hypothesis import given, settings, strategies as st

from autotest.bms import BatteryManagementSystem, BMSFault, max_charge_current
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import InstrumentCluster, WheelSpeedECU
from autotest.sensors import SensorFault, TemperatureSensor

pytestmark = pytest.mark.robustness

# ---- 1. sensor fault injection: wrap a healthy source and corrupt it -----
class FaultyADC:
    """Decorator around any ADC that injects classic automotive sensor faults."""

    def __init__(self, inner, mode, after=0):
        self.inner, self.mode, self.after, self.n, self.last = inner, mode, after, 0, None

    def read(self):
        value = self.inner.read()
        self.n += 1
        if self.n <= self.after:
            self.last = value
            return value
        if self.mode == "stuck":           # stuck-at last good value
            return self.last
        if self.mode == "short_gnd":
            return 0
        if self.mode == "open":
            return 4095
        raise AssertionError(self.mode)

class ConstADC:
    def __init__(self, v):
        self.v = v
```

```

    def read(self):
        return self.v

@pytest.mark.parametrize("mode, message", [("short_gnd", "ground"), ("open", "open
circuit")])
def test_hard_faults_are_reported_not_converted_to_a_temperature(mode, message):
    sensor = TemperatureSensor(FaultyADC(ConstADC(2000), mode, after=2))
    sensor.read(), sensor.read()
    with pytest.raises(SensorFault, match=message):
        sensor.read()

def test_stuck_sensor_is_NOT_detected_today_documenting_a_gap():
    sensor = TemperatureSensor(FaultyADC(ConstADC(2000), "stuck", after=1))
    values = {sensor.read() for _ in range(50)}
    assert len(values) == 1  # a stuck value looks perfectly healthy -> a plausibility check
is missing

# ---- 2. bus fault injection with a seeded RNG (reproducible!) -----
@pytest.mark.parametrize("seed", range(5))
def test_cluster_never_displays_an_invented_speed_under_bit_errors(seed, clock):
    rng = random.Random(seed)
    bus = VirtualCANBus()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)

    def flip_a_bit_sometimes(frame):
        if rng.random() < 0.3:
            data = bytearray(frame.data)
            data[rng.randrange(len(data))] ^= 1 << rng.randrange(8)
            return CANFrame(frame.arbitration_id, bytes(data))
        return frame

    bus.add_fault(flip_a_bit_sometimes)
    sent = set()
    for i in range(200):
        speed = float(rng.randrange(0, 250))
        sent.add(f"{speed:.0f}")
        ecu.send_speed(speed)
        clock.advance(0.01)
        shown = cluster.displayed_speed()
        assert shown == "--" or shown in sent, f"frame {i}: displayed invented value {shown}"

def test_duplicate_frames_are_rejected_by_alive_counter(clock):
    bus = VirtualCANBus()
    cluster = InstrumentCluster(bus, clock)
    ecu = WheelSpeedECU(bus)
    ecu.send_speed(50)
    duplicate = bus.log[-1]
    bus.send(duplicate)  # replayed frame, same counter
    assert cluster.diag._dtcs["U0100"].fail_count == 1

# ---- 3. fuzzing the diagnostic interface: random bytes in, SANE bytes out -----
@settings(max_examples=500, deadline=None)
@given(st.binary(max_size=20))
@pytest.mark.requirement("REQ-DIAG-002")
def test_uds_handler_never_raises_and_always_answers_with_a_valid_response(request_bytes):
    response = DiagnosticManager().handle_request(request_bytes)
    assert isinstance(response, bytes) and response
    assert response[0] in (0x7F, 0x59, 0x54, 0x62)
    if response[0] == 0x7F:
        assert len(response) == 3

```

```

@pytest.mark.requirement("REQ-DIAG-002")
@given(st.integers(0, 255).filter(lambda s: s not in (0x14, 0x19, 0x22)),
st.binary(max_size=6))
def test_unknown_service_gets_negative_response_0x11(service, rest):
    assert DiagnosticManager().handle_request(bytes([service]) + rest) == bytes([0x7F,
service, 0x11])

def test_seeded_random_fuzz_is_reproducible_in_ci():
    """Plain-pytest fuzzing: seed printed on failure -> anyone can replay it."""
    rng = random.Random(1234)
    mgr = DiagnosticManager()
    for _ in range(2000):
        blob = bytes(rng.randrange(256) for _ in range(rng.randrange(0, 12)))
        mgr.handle_request(blob)

# ---- 4. hostile numeric input: NaN / inf -----
@pytest.mark.parametrize("bad", [math.inf, -math.inf])
def test_infinite_temperature_is_handled_safely(bad):
    assert max_charge_current(50, bad) == 0.0

@pytest.mark.xfail(strict=True, reason="BUG-201: NaN temperature falls through every
comparison -> FULL charge current")
def test_nan_temperature_must_not_allow_charging():
    try:
        current = max_charge_current(50, math.nan)
    except ValueError: # rejecting the input is as safe as returning 0 A
        return
    assert current == 0.0

@pytest.mark.xfail(strict=True, reason="BUG-202: NaN cell voltage is never > max or < min ->
reported healthy")
def test_nan_cell_voltage_must_raise_a_fault_or_error():
    try:
        fault = BatteryManagementSystem().evaluate([3.7, math.nan, 3.7], 25)
    except ValueError:
        return
    assert fault is not BMSFault.NONE

# ---- 5. recovery: the system must come back after the fault is gone -----
def test_dtc_heals_after_enough_good_cycles():
    d = DiagnosticManager(fail_threshold=2, heal_threshold=3)
    for _ in range(2):
        d.report("U0121", failed=True)
    assert d.active_codes() == ["U0121"]
    for _ in range(3):
        d.report("U0121", failed=False)
    assert d.active_codes() == [] and d.stored_codes() == ["U0121"] # history is kept

```

Exercise skeleton — source

labs/lab08_robustness/exercise_lab08.py

```

"""Lab 8 exercises. Run: pytest labs/lab08_robustness/exercise_lab08.py -v"""
import pytest

# Exercise 1 - FIX BUG-201 and BUG-202 in src/autotest/bms.py (reject NaN with ValueError).
# Then REMOVE the xfail markers in test_lab08_robustness.py and watch the suite stay green.
# (strict xfail goes RED when the bug is fixed - that is the feature: it forces cleanup.)
def test_bugs_fixed():

```

```

    pytest.fail("TODO: make the two strict-xfail tests pass for real")

    # Exercise 2 - Add a plausibility check to TemperatureSensor.read_filtered: a value that does
    # not change AT ALL for 100 consecutive reads raises SensorFault('stuck'). Write the failing
    # test first (TDD), then implement.
    def test_stuck_sensor_detected():
        pytest.fail("TODO")

    # Exercise 3 - Write a fault hook for VirtualCANBus that models a *burst error*: 5
consecutive
    # frames lost. Show the cluster blanks the display and later recovers, and that the DTC
history
    # is retained. Use FakeClock - no sleeping.
    def test_burst_loss_and_recovery(bus, clock):
        pytest.fail("TODO")

```

Lab 9 — Regression, golden-file and log-replay testing

Duration: 2.5 h · **Type:** regression / snapshot / replay · **Code under test:** `InstrumentCluster`, `DiagnosticManager`

Why it matters

Software changes continuously over a 7-year programme; every release must not re-break what was fixed. Replaying **recorded drive cycles** (MDF/ASC logs) against new builds is how OEMs catch regressions that no one thought to specify.

Learning objectives

Write bug-guard tests named after tickets; compare complex output with **golden files**; update goldens deliberately (`--update-golden`) and review the diff; replay a recorded CAN log through the system.

The workflow

1. A defect is reported → write a **failing** test that reproduces it.
2. Fix → test passes → keep it forever (named `test_bug_NNN_...`, docstring says what broke).
3. Complex outputs (reports, transcripts, dashboards over a drive) → golden file.
4. Changing a golden file is a **reviewed code change**, never a reflex.

Run it

```
pytest labs/lab09_regression_golden -v
python tools/make_drive_log.py          # regenerate the recorded drive cycle
pytest labs/lab09_regression_golden --update-golden # only after reading the diff!
```

The finding behind this lab: BUG-130

Replaying `data/drive_cycle.log` (which contains one corrupted frame) showed the cluster discarding **two** frames, then setting DTC U0100: the CRC-failed frame did not advance the expected alive counter. The fix is in `ecu.py`; the guard is `test_bug_130_...`. Revert the fix (3 lines) and 3 tests go red — try it.

Exercises — `exercise_lab09.py`

Triage BUG-120 (reproduce first — the report may be wrong!) · golden transcript of a UDS session · mutate the log and read the diff.

Debrief

- What is the danger of `--update-golden` in a hurry? How does code review mitigate it?
- A golden file contains a timestamp. What happens in CI? How do you fix it?
- Where should recorded logs come from, and how do you keep them valid when the interface evolves?

Worked example — source

labs/lab09_regression_golden/test_lab09_regression_golden.py

```
"""Lab 9 - regression testing: bug-guard tests, golden files and log replay.

Rule: every defect that ever escaped gets a test BEFORE it is fixed. Golden files
protect complex outputs (reports, logs) that are tedious to assert field-by-field.

Create/refresh golden files deliberately with:  pytest labs/lab09_regression_golden --update-
golden
(then READ the git diff - an updated golden file is a reviewed change, not a formality)
"""
from pathlib import Path

import pytest

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.clock import FakeClock
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import InstrumentCluster, WheelSpeedECU

pytestmark = pytest.mark.regression
DATA = Path(__file__).parent / "data"

# ---- 1. bug-guard tests: name them after the ticket -----
---
def test_bug_087_alive_counter_wrap_15_to_0_was_rejected(clock):
    """BUG-087: after 16 frames the 4-bit counter wrapped and the cluster went blank."""
    bus = VirtualCANBus()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)
    for i in range(40):
        ecu.send_speed(i)
        assert cluster.displayed_speed() == str(i), f"frame {i}"

def test_bug_093_crc_error_must_not_update_last_rx_time(clock):
    """BUG-093: garbage frames kept the timeout from expiring (cluster showed stale
speed)."""
    bus = VirtualCANBus()
    cluster = InstrumentCluster(bus, clock)
    WheelSpeedECU(bus).send_speed(80)
    for _ in range(5):
        clock.advance(0.05)
        bus.send(CANFrame(0x1A0, b"\x00\x01\x02\x03\x04")) # invalid CRC
    assert cluster.displayed_speed() == "--"

# ---- 2. golden file for a text report -----
-----
def test_dtc_report_matches_golden(golden):
    d = DiagnosticManager(fail_threshold=2, heal_threshold=2)
    for _ in range(2):
        d.report("U0121", failed=True) # confirmed
```

```

d.report("P0300", failed=True)          # pending only
for _ in range(2):
    d.report("U0100", failed=True)
for _ in range(2):
    d.report("U0100", failed=False)      # confirmed then healed
golden("dtc_report.txt", d.export_report())

# ---- 3. replaying a recorded drive cycle -----
-----
def replay(path: Path):
    """Feed a recorded log into a fresh cluster; return what the driver would have seen."""
    clock, bus = FakeClock(), VirtualCANBus()
    cluster = InstrumentCluster(bus, clock)
    seen = []
    for line in path.read_text().splitlines():
        t, can_id, data = line.split()
        clock.advance(float(t) - clock.now())
        bus.send(CANFrame(int(can_id, 16), bytes.fromhex(data)))
        seen.append(f"{float(t):6.3f}s {cluster.displayed_speed():>3}")
    return "\n".join(seen) + "\n", cluster

def test_recorded_drive_cycle_shows_the_same_dashboard_as_last_release(golden):
    transcript, _ = replay(DATA / "drive_cycle.log")
    golden("drive_cycle_display.txt", transcript)

def test_replay_of_the_recording_raises_no_dtc():
    _, cluster = replay(DATA / "drive_cycle.log")
    assert cluster.diag.stored_codes() == [] # one recorded bit error is only debounced

def test_bug_130_single_bit_error_must_cost_exactly_one_frame(clock):
    """BUG-130 (found by the replay in this lab): a CRC error left the expected alive counter
    unchanged, so the NEXT good frame failed the counter check too -> two losses, and with
    fail_threshold=2 a DTC for what was a single bit error."""
    bus = VirtualCANBus()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)
    ecu.send_speed(10)
    flip = lambda f: CANFrame(f.arbitration_id, bytes([f.data[0] ^ 1]) + f.data[1:])
    bus.add_fault(flip)
    ecu.send_speed(20)          # corrupted in flight
    bus.clear_faults()
    ecu.send_speed(30)          # must be accepted immediately
    assert cluster.displayed_speed() == "30"
    assert cluster.diag.stored_codes() == []

```

Exercise skeleton — source

labs/lab09_regression_golden/exercise_lab09.py

```

"""Lab 9 exercises. Run: pytest labs/lab09_regression_golden/exercise_lab09.py -v"""
import pytest

# Exercise 1 - You receive bug report BUG-120: "cluster shows 0 instead of -- when the ECU
# sends valid=0 (speed invalid)". Reproduce it with a failing test FIRST, then fix
# src/autotest/ecu.py if needed. (Hint: read InstrumentCluster._on_frame carefully.)
def test_bug_120_invalid_flag():
    pytest.fail("TODO")

# Exercise 2 - Golden file for UDS: create golden/uds_session.txt that records requests and
# responses (hex) for: read DTCs, read VIN, clear DTCs, read DTCs again, unknown service.

```

```
def test_uds_session_transcript(golden):  
    pytest.fail("TODO")  
  
    # Exercise 3 - Edit data/drive_cycle.log by hand (change one speed). Run the Lab 9 tests and  
    # read the  
    # diff produced by pytest. Revert via git. What is the danger of blindly running --update-  
    # golden?
```

Lab 10 — Performance and real-time budget testing

Duration: 2.5 h · **Type:** performance / timing / memory · **Code under test:** `CruiseController.control`, `codec`, `LeakDetector`, `VirtualCANBus`

Why it matters

A control task that takes 12 ms in a 10 ms cycle is a functional failure. A decoder that cannot keep up with a saturated 500 kbit/s bus drops frames. A log that grows forever crashes the gateway after 3 days of driving.

Learning objectives

Build a measurement helper (warm-up, repeats, GC off, percentiles); express **budgets** instead of averages; test **scaling** (does cost grow with history?) with ratios, not wall-clock numbers; check **memory** with `tracemalloc`; keep performance tests from making CI flaky.

Rules for non-flaky performance tests

1. Assert on **p99 / max** versus a **generous** budget (10× headroom), never on a mean.
2. Prefer **relative** assertions (ratio between sizes) over absolute milliseconds.
3. Warm up, repeat, disable GC while measuring.
4. Run in a separate CI stage (`pytest -m performance`) so parallel jobs do not disturb it.
5. Real worst-case execution time is measured **on target hardware** — host timing only catches algorithmic regressions.

Run it

```
pytest labs/lab10_performance -v
pytest labs -m "not performance"      # exclude from the everyday loop
pytest --durations=10                 # the suite's own slowest tests
```

Two real scalability defects (exercises)

- `LeakDetector.add` recomputes `max()` over the whole 60 s window → **O(window)** per sample; at 1 kHz that is 60 000 elements per call.
- `VirtualCANBus.log` is unbounded → memory grows without limit.

The instructor solutions demonstrate both (ratio > 3 before, < 3 after; bounded deque).

Exercises — `exercise_lab10.py`

Scaling test + monotonic-deque fix · `tracemalloc` + bounded log · a `budget(seconds)` context manager.

Debrief

- Why is `time.time()` the wrong clock? Why disable GC?
- Your p99 is fine but max is 40× larger. Is that a problem for a hard real-time task? For a soft one?
- How would you move this test to the target ECU?

Worked example — source

`labs/lab10_performance/test_lab10_performance.py`

```
"""Lab 10 - performance and real-time budgets.

Rules for non-flaky performance tests in CI:
* assert on percentiles / worst cases against GENEROUS budgets (10x headroom), never means
* use time.perf_counter, warm up first, and repeat
* prefer RELATIVE assertions (scaling, ratio to a baseline) over absolute wall-clock
numbers
* run them in a separate stage: pytest -m performance / pytest -m "not performance"
"""
import gc
import statistics
import time
import tracemalloc

import pytest

from autotest.can import CANFrame, CANMessage, Signal, crc8, e2e_check, e2e_protect
from autotest.cruise import CruiseController
from autotest.ecu import WHEEL_SPEED_MSG

pytestmark = pytest.mark.performance

def measure(fn, repeat=2000, warmup=200):
    """Return per-call durations in seconds."""
    for _ in range(warmup):
        fn()
    gc.disable() # avoid collector pauses polluting the sample
    try:
        out = []
        for _ in range(repeat):
            t0 = time.perf_counter()
            fn()
            out.append(time.perf_counter() - t0)
    finally:
        gc.enable()
    return out

def p(samples, pct):
    return statistics.quantiles(samples, n=1000)[int(pct * 10) - 1]

# ---- 1. real-time budget: the control task must fit its 10 ms cycle with margin -----
--
```

```

@pytest.mark.requirement("REQ-CC-003")
def test_cruise_control_step_meets_its_10ms_cycle_with_big_margin():
    cc = CruiseController()
    cc.power_on()
    cc.set(100)
    samples = measure(lambda: cc.control(97.0, 0.01))
    budget = 0.010 * 0.10 # use at most 10 % of the cycle time (1 ms)
    assert p(samples, 99) < budget, f"p99={p(samples, 99) * 1e6:.1f} us"
    assert max(samples) < 0.010, "worst case must never miss the cycle deadline"

# ---- 2. throughput: can we keep up with a saturated 500 kbit/s bus? -----
----
def test_decoder_keeps_up_with_a_saturated_500kbit_bus():
    """A bus at 500 kbit/s carries at most ~4 500 frames/s (111-bit worst-case frame)."""
    frames = [e2e_protect(WHEEL_SPEED_MSG.encode({"speed_kmh": i % 300, "valid": 1}), i) for
i in range(4500)]
    t0 = time.perf_counter()
    for i, raw in enumerate(frames):
        status, payload = e2e_check(raw, i)
        WHEEL_SPEED_MSG.decode(payload)
    elapsed = time.perf_counter() - t0
    assert elapsed < 0.5, f"decoded 1 s of worst-case bus traffic in {elapsed:.3f} s"

# ---- 3. scaling: cost must not grow with history length -----
-----
def test_crc_cost_scales_linearly_with_length():
    short, long_ = bytes(8), bytes(8 * 50)
    t_short = statistics.median(measure(lambda: crc8(short), repeat=500))
    t_long = statistics.median(measure(lambda: crc8(long_), repeat=500))
    ratio = t_long / t_short
    assert ratio < 50 * 3, f"50x more data cost {ratio:.0f}x more time (expected ~50x,
linear)"

# ---- 4. memory: tracemalloc -----
-----
def test_encoding_does_not_leak_memory():
    sig = CANMessage(1, "m", 2, (Signal("v", 0, 16, 0.1),))
    sig.encode({"v": 1.0}) # warm caches
    tracemalloc.start()
    before = tracemalloc.take_snapshot()
    for i in range(20000):
        sig.encode({"v": (i % 1000) * 0.1})
    after = tracemalloc.take_snapshot()
    tracemalloc.stop()
    growth = sum(s.size_diff for s in after.compare_to(before, "filename"))
    assert growth < 200_000, f"retained {growth} bytes after 20 000 encodes"

# ---- 5. finding the slow tests of the suite itself -----
-----
def test_timing_is_part_of_the_report():
    """Run `pytest --durations=10` to list the slowest tests - a free performance monitor."""
    assert True

```

Exercise skeleton — source

labs/lab10_performance/exercise_lab10.py

```

"""Lab 10 exercises. Run: pytest labs/lab10_performance/exercise_lab10.py -v"""
import time

import pytest

```

```

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.tpms import LeakDetector

# Exercise 1 - SCALING BUG: LeakDetector.add() recomputes max() over its whole 60 s window.
# At 1 kHz sampling the window holds ~60 000 samples. Write a test that feeds 1 000 and then
# 20 000 samples (all inside the window) and compares the per-call time. Show the ratio.
# Then fix it with a monotonic deque (O(1) amortised) and prove the ratio shrinks.
def test_leak_detector_cost_does_not_grow_with_window_fill():
    pytest.fail("TODO")

# Exercise 2 - MEMORY BUG: VirtualCANBus.log keeps every frame forever. Use tracemalloc to
show
# 200 000 sends retain tens of MB. Design the fix (bounded deque with maxlen? opt-in
logging?)
# and justify it in a comment. Keep Lab 5 tests green.
def test_bus_log_is_bounded():
    pytest.fail("TODO")

# Exercise 3 - Write a "budget" fixture that wraps a block in perf_counter and fails the test
# if it takes longer than N seconds: `with budget(0.05): ...`. Use contextlib.contextmanager.
def test_budget_fixture():
    pytest.fail("TODO")

```

Lab 11 — Software-in-the-loop (SIL) simulation and acceptance criteria

Duration: 3 h · **Type:** closed-loop system testing / model-based testing · **Code under test:**

CruiseController + Vehicle, ABSController + QuarterCar

Why it matters

Controllers are only "correct" in a loop: the throttle value matters because of what the car then does. SIL runs the production control code against a plant model long before a prototype exists; the same scenarios later run on HIL and in the car.

Learning objectives

Turn requirements into **measurable acceptance criteria** (overshoot, settling time, dip, steady-state error, stopping distance); **test the model too** (a wrong plant gives false confidence); use a *control group* (the scenario without ABS must fail); run scenario sweeps; detect a controller whose tuning fails a requirement.

Metrics used

Metric	Definition	Criterion
Overshoot	(peak – target) / step	< 20 % (< 4 km/h on a 20 km/h step)
Settling time	last time outside ± 1 km/h	< 25 s
Hill dip (5 %)	target – minimum after grade starts	< 4 km/h, full recovery
Steady-state error	after 120 s	< 0.1–0.5 km/h
ABS lock time	seconds with slip > 0.95 while $v \geq 5$ m/s	= 0
ABS distance	vs locked-wheel braking	≥ 10 % shorter

Run it

```
pytest labs/lab11_sil_simulation -v
pytest -m sil
```

Guided tour

- **Plant sanity tests first:** coasting, top speed, grade — if the model is wrong, everything downstream is noise.

- `test_baseline_without_abs_locks_the_wheel` — proves the scenario is hazardous, so "ABS prevents lock" is meaningful.
- Determinism test: fixed-step simulation $\Rightarrow$ identical results every run (a prerequisite for regression testing).

Exercises — `exercise_lab11.py`

8 % hill criteria (default tuning **fails** — find gains that pass) · ABS on ice · a 5 km/h biased speed sensor (what happens, what would detect it?).

Debrief

- Your simulation passes. What *model assumptions* could still make the real car fail?
- Which scenarios would you add for a safety case (cut-in vehicle, sensor dropout mid-brake)?
- When does a SIL result transfer to HIL, and what must be re-validated?

Worked example — source

labs/lab11_sil_simulation/test_lab11_sil_simulation.py

```

"""Lab 11 - software-in-the-loop (SIL): the controller runs against a simulated plant.

Open-loop unit tests say "the function returns 0.7". Closed-loop tests say "the CAR reaches
100 km/h without overshooting more than 4 km/h". Acceptance criteria are written as
measurable numbers.
"""
import pytest

from autotest.abs import ABSController, Valve, QuarterCar, simulate_braking, slip_ratio,
tyre_mu
from autotest.cruise import CruiseController, CruiseState
from autotest.vehicle import Vehicle, run_cruise

pytestmark = [pytest.mark.sil, pytest.mark.system]

# ----- helpers: metrics -----
def settling_time(trace, target, band=1.0):
    """First time after which speed stays within +/- band of the target."""
    last_bad = max((t for t, v, _ in trace if abs(v - target) > band), default=-1.0)
    return last_bad + (trace[1][0] - trace[0][0])

def overshoot_pct(trace, target, start):
    return max(0.0, (max(v for _, v, _ in trace) - target) / (target - start) * 100)

@pytest.fixture
def cruise_at_100():
    cc = CruiseController()
    cc.power_on()
    cc.set(100)
    return cc

# ----- plant sanity (test the

```

```

model!) ----
def test_plant_coasts_down_without_throttle():
    car = Vehicle(speed_ms=30)
    for _ in range(600):
        car.step(0.0, 0.1)
    assert 0 < car.speed_ms < 30

def test_plant_cannot_exceed_physical_top_speed():
    car = Vehicle()
    for _ in range(20000):
        car.step(1.0, 0.1)
    assert 200 < car.speed_kmh < 300 # drag balance

def test_plant_steep_grade_slows_the_car():
    flat, hill = Vehicle(speed_ms=25), Vehicle(speed_ms=25)
    for _ in range(100):
        flat.step(0.2, 0.1, 0.0)
        hill.step(0.2, 0.1, 0.08)
    assert hill.speed_ms < flat.speed_ms

# ----- cruise acceptance -----
def test_reaches_target_from_80_with_acceptable_dynamics(cruise_at_100):
    trace = run_cruise(cruise_at_100, start_kmh=80, duration_s=120)
    assert overshoot_pct(trace, 100, 80) < 20.0 # i.e. below 4 km/h on a 20 km/h step
    assert settling_time(trace, 100, band=1.0) < 25.0
    assert abs(trace[-1][1] - 100) < 0.1 # zero steady-state error (integral
action)

def test_holds_speed_when_a_5_percent_hill_starts(cruise_at_100):
    trace = run_cruise(cruise_at_100, 100, 120, grade_profile=lambda t: 0.05 if t >= 20 else
0.0)

    assert min(v for t, v, _ in trace if t >= 20) > 96.0 # dip no larger than 4 km/h
    assert abs(trace[-1][1] - 100) < 0.5 # and fully recovered

def test_throttle_never_leaves_0_1_even_for_huge_errors():
    cc = CruiseController()
    cc.power_on()
    cc.set(180)
    trace = run_cruise(cc, 30, 60)
    assert all(0.0 <= u <= 1.0 for _, _, u in trace)

@pytest.mark.requirement("REQ-CC-002")
def test_brake_event_in_the_loop_hands_back_control(cruise_at_100):
    brake_at_30s = {30.0: lambda cc, car: cc.brake()}
    trace = run_cruise(cruise_at_100, 100, 40, events=brake_at_30s)
    assert cruise_at_100.state is CruiseState.STANDBY
    assert all(u == 0.0 for t, _, u in trace if t > 30)

def test_resume_brings_the_car_back_to_the_saved_speed(cruise_at_100):
    ev = {20.0: lambda cc, car: cc.cancel(), 40.0: lambda cc, car: cc.resume(car.speed_kmh)}
    trace = run_cruise(cruise_at_100, 100, 120, events=ev)
    mid = next(v for t, v, _ in trace if t == 39.0)
    assert mid < 100 # slowed down while cancelled
    assert abs(trace[-1][1] - 100) < 0.5 # and came back

@pytest.mark.parametrize("target", [40, 70, 100, 130, 160])
def test_converges_for_every_target_in_the_envelope(target):
    cc = CruiseController()
    cc.power_on()

```

```

cc.set(target)
trace = run_cruise(cc, target - 10, 200)
assert abs(trace[-1][1] - target) < 0.5

# ----- ABS acceptance -----
-----

def test_tyre_model_peaks_at_20_percent_slip():
    assert max(tyre_mu(s / 100) for s in range(101)) == pytest.approx(1.0)
    assert tyre_mu(0.2) > tyre_mu(1.0)

@pytest.mark.parametrize("vehicle, wheel, expected", [
    (30, 30, Valve.APPLY),          # rolling freely
    (30, 21, Valve.HOLD),           # 30 % slip? -> see below
    (30, 20, Valve.RELEASE),
    (3, 0, Valve.APPLY),            # below 5 m/s ABS stays out of the way
])
def test_abs_decisions(vehicle, wheel, expected):
    actual = ABSController().decide(vehicle, wheel)
    if (vehicle, wheel) == (30, 21):          # 30 % slip is above the 25 % release
        expected = Valve.RELEASE
    assert actual is expected

@pytest.mark.requirement("REQ-ABS-001")
def test_wheel_does_not_lock_while_abs_is_active():
    result = simulate_braking(use_abs=True)
    assert result["locked_s"] == 0.0
    assert result["peak_slip"] < 0.35

@pytest.mark.requirement("REQ-ABS-002")
def test_abs_shortens_stopping_distance_by_at_least_10_percent():
    with_abs, without = simulate_braking(True), simulate_braking(False)
    assert with_abs["distance_m"] < 0.9 * without["distance_m"]

def test_baseline_without_abs_locks_the_wheel():
    """The 'control group': proves the scenario really is dangerous, so the ABS test means
    something."""
    assert simulate_braking(use_abs=False)["locked_s"] > 1.0

def test_simulation_is_deterministic():
    assert simulate_braking(True) == simulate_braking(True)

```

Exercise skeleton — source

labs/lab11_sil_simulation/exercise_lab11.py

```

"""Lab 11 exercises. Run: pytest labs/lab11_sil_simulation/exercise_lab11.py -v"""
import pytest

# Exercise 1 - Write acceptance criteria as numbers: "dip <= 4 km/h and recovery within 20 s"
for a
# 8 % hill. Run the simulation. Does the default controller (kp=0.05, ki=0.01) pass?
# If not, tune kp/ki (write the sweep as a parametrized test) and state your choice.
def test_8_percent_hill():
    pytest.fail("TODO")

# Exercise 2 - Low-friction surface: change tyre_mu's peak to 0.3 (monkeypatch). With ABS the

```

```
# stopping distance must be < locked-wheel distance. Does the 10 % improvement still hold?
def test_ice():
    pytest.fail("TODO")

# Exercise 3 - Sensor-fault in the loop: feed the controller a speed that is 5 km/h too low
# (miscalibrated wheel sensor). Predict, then measure, the steady-state ERROR of the real
car.

# Which requirement/diagnostic would catch this in the field?
def test_biased_speed_sensor():
    pytest.fail("TODO")
```

Lab 12 — Test adequacy: coverage and mutation testing

Duration: 2.5 h · **Type:** tests of the tests · **Tools:** `pytest-cov`, `tools/mini_mutate.py`

Why it matters

ISO 26262 asks for structural coverage (statement → branch → MC/DC by ASIL). But **coverage only proves code ran, not that anything checked it**. Mutation testing asks the real question: *if the code were wrong, would a test fail?*

Learning objectives

Measure line/branch coverage and read the `term-missing` report; build a suite with 100 % coverage that is nearly worthless; run a mutation tester and classify survivors as **weak test** or **equivalent mutant**; understand the cost/benefit of both metrics.

Run it

```
pytest labs/lab12_coverage_mutation/weak_suite.py \
    --cov=autotest.bms --cov-branch --cov-report=term-missing      # full coverage, weak
assertions
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current --
tests labs/lab12_coverage_mutation/weak_suite.py
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current --
tests labs/lab03_black_box/test_lab03_black_box.py
python tools/mini_mutate.py ... --list                            # just list the mutants
pytest labs/lab12_coverage_mutation -v                          # automated assertions about both scores
(slow, ~25 s)
```

What to expect

Suite	Line+branch coverage of the function	Mutation score
<code>weak_suite.py</code>	100 %	≈ 4 % (1/25)
Lab 3 suite	100 %	22/25 — all three survivors are equivalent mutants

Equivalent mutants: `soc >= 100 → soc > 100` (or `>= 101`) and `soc > 80 → soc >= 80` change the code but not the behaviour — the taper formula already yields 0 A at 100 % and the full limit at 80 %. No test can kill them. They are not noise: the explicit `soc >= 100` guard is **redundant code**, which a reviewer may now delete or document. Recognising equivalent mutants is a human judgement call.

A tool bug worth knowing. The first version of `mini_mutate.py` gave *non-deterministic* scores. Cause: Python validates cached bytecode by source mtime (whole seconds) and file size, so two same-length mutants written within one second (100 → 101) silently reused the previous mutant's `.pyc`. Fix: `PYTHONDONTWRITEBYTECODE=1` in the test subprocess. Lesson: when a test tool disagrees with itself, suspect the environment before the tests.

How `mini_mutate.py` works (read it — it is ~120 lines)

Parses the target with `ast`, enumerates mutation sites (comparison/arithmetic/boolean operators, numeric constants), rewrites one site at a time in a scratch copy of the project, runs pytest, and counts a mutant as *killed* if the run fails.

Exercises — `exercise_lab12.py` (terminal work)

Find the lowest-branch-coverage module · classify survivors of `CruiseController.control` (state-machine suite: 8/12; + SIL suite: 10/12 — which exact PI arithmetic is still unchecked?) · write one test that kills the most survivors in `DiagnosticManager.report` · argue why 100 % mutation score is not a sensible CI gate.

Debrief

- Can you reach 100 % branch coverage with a test that has no assertions? (Open `weak_suite.py`.)
- Why is MC/DC stricter than branch coverage, and which classes of defect does it target?
- Where would you spend a limited test budget: raising coverage 90→100 %, or mutation score 60→80 % on the safety core?

Worked example — source

labs/lab12_coverage_mutation/test_lab12_coverage_mutation.py

```
"""Lab 12 - coverage is necessary, not sufficient: mutation analysis as a test of the tests.

These tests drive tools/mini_mutate.py (a subprocess per mutant, so they are marked slow).
"""
import inspect
import json
import subprocess
import sys
from pathlib import Path

import pytest

from autotest.bms import max_charge_current

ROOT = Path(__file__).resolve().parents[2]
pytestmark = pytest.mark.slow

def mutation_score(tests: str) -> tuple[int, int, str]:
```

```

        out = subprocess.run(
            [sys.executable, str(ROOT / "tools" / "mini_mutate.py"),
             "--target", "src/autotest/bms.py", "--function", "max_charge_current", "--tests",
tests],
            cwd=ROOT, capture_output=True, text=True, timeout=300,
        ).stdout
        line = next(l for l in out.splitlines() if l.startswith("mutation score"))
        killed, total = line.split(":")[1].split("=")[0].strip().split("/")
        return int(killed), int(total), out

def test_the_strong_suite_from_lab3_kills_all_non_equivalent_mutants():
    killed, total, out = mutation_score("labs/lab03_black_box/test_lab03_black_box.py")
    survivors = total - killed
    # The three survivors are EQUIVALENT mutants - they change the code but not its
behaviour:
    # `soc >= 100` -> `soc > 100` / `soc >= 101`, and `soc > 80` -> `soc >= 80`.
    # The taper formula already yields 0 A at 100 % and the full limit at 80 %, so the
explicit
    # `soc >= 100` guard is REDUNDANT code - mutation testing just told us so.
    # No test can kill an equivalent mutant; recognising them is a human skill.
    assert survivors <= 3, out

def test_the_weak_suite_has_full_coverage_but_a_poor_mutation_score():
    killed, total, out = mutation_score("labs/lab12_coverage_mutation/weak_suite.py")
    assert killed / total < 0.6, out

def test_coverage_report_for_bms_is_complete_for_the_weak_suite(tmp_path):
    report = tmp_path / "cov.json"
    r = subprocess.run(
        [sys.executable, "-m", "pytest", "labs/lab12_coverage_mutation/weak_suite.py", "-q",
         "--cov=autotest.bms", "--cov-branch", f"--cov-report=json:{report}", "-p",
"no:cacheprovider"],
        cwd=ROOT, capture_output=True, text=True,
    )
    assert r.returncode == 0, r.stdout
    data = json.loads(report.read_text())["files"]["src/autotest/bms.py"]
    lines, first = inspect.getsourcelines(max_charge_current)
    inside = set(range(first, first + len(lines)))
    # The weak suite never touches the BatteryManagementSystem class (expected). What matters
is
    # that NOTHING inside max_charge_current is missing - neither lines nor branches.
    assert not inside & set(data["missing_lines"]), data["missing_lines"]
    assert not inside & {a for a, _ in data["missing_branches"]}, data["missing_branches"]

```

Additional file

labs/lab12_coverage_mutation/weak_suite.py

```

"""A deliberately BAD test suite: 100 % line and branch coverage of max_charge_current,
yet it would not notice most defects. (Not collected by default: name is not test_*.)

pytest labs/lab12_coverage_mutation/weak_suite.py --cov=autotest.bms --cov-branch --cov-
report=term-missing
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current \
    --tests labs/lab12_coverage_mutation/weak_suite.py
"""
from autotest.bms import max_charge_current

def test_runs_every_branch_but_checks_almost_nothing():
    for soc, temp in [(50, -5), (50, 60), (50, 5), (50, 25), (100, 25), (90, 25)]:
        result = max_charge_current(soc, temp)

```

```

        assert result is not None
        assert result >= 0          # true for almost any mutant too

def test_invalid_input_path_is_executed_too():
    try:
        max_charge_current(101, 25)
    except ValueError:
        pass                        # executed, never verified that the right error is
raised

```

Exercise skeleton — source

labs/lab12_coverage_mutation/exercise_lab12.py

```

"""Lab 12 exercises (run in a terminal, answers go in comments / a short report).

1. Measure: pytest labs/lab02_pytest labs/lab03_black_box --cov=autotest --cov-branch --cov-
report=term-missing
    Which module has the lowest BRANCH coverage? Write 2 tests to raise it - and say why 100 %
    is not the goal for every module.
2. Mutate: python tools/mini_mutate.py --target src/autotest/cruise.py --function control \
    --tests labs/lab06_state_machine/test_lab06_state_machine.py
    (expect 8/12). Add the simulation suite: --tests
labs/lab06_state_machine/test_lab06_state_machine.py labs/lab11_sil_simulation
    (expect 10/12). Two PI-arithmetic mutants STILL survive: even closed-loop tests only check
    behaviour in broad strokes. Classify each survivor: (a) weak test -> write a unit test
with an
    exact hand-computed throttle value for CruiseController(kp=0.05, ki=0.01) that kills them,
    or (b) equivalent mutant -> explain why.
3. Mutate diagnostics: --target src/autotest/diagnostics.py --function report \
    --tests labs/lab02_pytest labs/lab08_robustness
    Two survivors remain (both in the healing counters). Write ONE new test that kills both.
    Which technique from Labs 3/7 finds it fastest?
4. Discuss: why is a mutation score of 100 % impossible/undesirable as a hard CI gate?
"""
import pytest

def test_placeholder():
    pytest.fail("TODO: complete the exercises above in a terminal")

```

Lab 13 — Acceptance tests, requirements traceability and SIL/HIL switching

Duration: 3 h · **Type:** acceptance / requirements-based testing · **Artefacts:** `docs/requirements.csv` , `--req-report`

Why it matters

Assessors (ISO 26262, ASPICE SWE.4–SWE.6) do not ask "do you have tests?" but "**show me which test verifies requirement X, and its result.**" Traceability — requirement $\nRightarrow$ test $\nRightarrow$ result — is the evidence package.

Learning objectives

Write acceptance scenarios in **Given / When / Then** form; link tests to requirement IDs with a custom marker; produce a traceability matrix; make the test suite **fail when a requirement has no test** or a test cites an unknown requirement; run the same test body on a simulator and (optionally) on hardware.

The mechanism (see `conftest.py`)

```
@pytest.mark.requirement("REQ-BMS-003")    # on a test, class or module (pytestmark)
```

- `pytest --req-report` prints each requirement → PASS/FAIL with its tests.
- Meta-tests scan the repository: every ID in `docs/requirements.csv` needs ≥ 1 verifying test; every marker must cite a known ID.
- `@pytest.fixture(params=["sil", pytest.param("hil", marks=pytest.mark.hil)])` — `hil` tests are **skipped unless** `--hil` , so one test body serves simulator and bench.

Run it

```
pytest labs/lab13_acceptance_traceability -v
pytest labs/solutions -m "not slow" -q --req-report    # the matrix
pytest labs/lab13_acceptance_traceability --hil        # BenchADC raises: shows what a
missing bench looks like
```

Exercises — `exercise_lab13.py`

1. **Requirements-driven development:** `REQ-TPMS-001/002` exist in the CSV but (without `solutions/`) nothing verifies them: the meta-test is red → write the tests → green.
2. An acceptance scenario for the dashboard during acceleration.
3. A second switchable back-end (`virtual` vs `pcan`) for the CAN tests.

Debrief

- A requirement has 14 tests, another has 1. Is the second one less verified? What would you ask?
- PASS in the matrix, but the test never asserts on the requirement's *numbers*. How would an assessor catch that? (Hint: Lab 12.)
- What must change in a HIL back-end that is *not* just "a different class"? (timing, nondeterminism, safe state on test abort.)

Worked example — source

labs/lab13_acceptance_traceability/test_lab13_acceptance_traceability.py

```
"""Lab 13 - acceptance testing, requirement traceability and hardware-in-the-loop (HIL)
switching.

ISO 26262 / ASPICE style evidence: every requirement is linked to >= 1 verifying test and
the report shows PASS/FAIL per requirement:

    pytest labs --req-report -q
"""
import csv
import re
import subprocess
import sys
from pathlib import Path

import pytest

from autotest.bms import BatteryManagementSystem, BMSFault
from autotest.can import CANFrame
from autotest.diagnostics import DiagnosticManager
from autotest.sensors import SensorFault, TemperatureSensor

ROOT = Path(__file__).resolve().parents[2]
REQUIREMENTS = {row["id"]: row for row in csv.DictReader((ROOT / "docs" /
"requirements.csv").open())}

#
=====
# A. Acceptance scenarios (Given / When / Then) - written in the customer's language
#
=====
@pytest.mark.requirement("REQ-CAN-001")
class TestFeatureFrameValidation:
    """Feature: the gateway refuses malformed identifiers."""

    @pytest.mark.parametrize("ident, extended, ok", [
        (0x000, False, True), (0x7FF, False, True), (0x800, False, False),
        (0x800, True, True), (0xFFFFFFFF, True, True), (0x20000000, True, False), (-1, False,
False),
    ])
    def test_scenario_identifer_ranges(self, ident, extended, ok):
        # Given a gateway that builds frames
        # When it is asked for identifier <ident> (standard or extended)
        # Then the frame exists only if the identifier fits the identifier field
        if ok:
            assert CANFrame(ident, is_extended=extended).arbitration_id == ident
        else:
            with pytest.raises(ValueError):
                CANFrame(ident, is_extended=extended)
```

```

@pytest.mark.requirement("REQ-BMS-003")
class TestFeatureOvervoltageProtection:
    def test_scenario_one_weak_cell_in_a_healthy_pack(self):
        # Given a 96-cell pack where every cell is at 3.9 V
        cells = [3.9] * 96
        bms = BatteryManagementSystem()
        assert bms.evaluate(cells, 25) is BMSFault.NONE
        # When ONE cell rises to 4.21 V
        cells[57] = 4.21
        # Then an overvoltage fault is raised
        assert bms.evaluate(cells, 25) is BMSFault.OVERVOLTAGE

    def test_scenario_cell_exactly_at_the_limit_is_still_allowed(self):
        assert BatteryManagementSystem().evaluate([4.20] * 4, 25) is BMSFault.NONE

@pytest.mark.requirement("REQ-DIAG-001")
class TestFeatureDtcDebouncing:
    @pytest.mark.parametrize("n", [1, 2, 3, 5])
    def test_scenario_code_is_confirmed_only_after_n_consecutive_failures(self, n):
        # Given a manager that confirms after n failures
        diag = DiagnosticManager(fail_threshold=n)
        # When the monitor fails n-1 times
        for _ in range(n - 1):
            diag.report("P0301", failed=True)
        # Then nothing is stored; one more failure confirms it
        assert diag.stored_codes() == []
        diag.report("P0301", failed=True)
        assert diag.stored_codes() == ["P0301"]

    def test_scenario_a_good_cycle_resets_the_failure_streak(self):
        diag = DiagnosticManager(fail_threshold=3)
        for outcome in (True, True, False, True, True):
            diag.report("P0301", failed=outcome)
        assert diag.stored_codes() == [] # failures were never CONSECUTIVE enough

#
=====
# B. Traceability checks - the test suite tests its own documentation
#
=====

def _markers_in_repo():
    found = {}
    pattern = re.compile(r'mark\.requirement\("(REQ-[A-Z]+\d{3})"\)')
    for folder in ("labs", "solutions"):
        for path in (ROOT / folder).rglob("*.py"):
            if path.name.startswith("exercise_"):
                continue
            for req in pattern.findall(path.read_text()):
                found.setdefault(req, set()).add(path.name)
    return found

def test_every_requirement_is_verified_by_at_least_one_test():
    verified = _markers_in_repo()
    missing = sorted(set(REQUIREMENTS) - set(verified))
    assert not missing, f"requirements without a verifying test: {missing}"

def test_no_test_refers_to_an_unknown_requirement():
    unknown = sorted(set(_markers_in_repo()) - set(REQUIREMENTS))
    assert not unknown, f"typo or deleted requirement? {unknown}"

def test_requirement_report_option_prints_a_matrix():
    r = subprocess.run(

```

```

        [sys.executable, "-m", "pytest", "labs/lab05_integration", "-q", "--req-report", "-p", "no:cacheprovider"],
        cwd=R00T, capture_output=True, text=True,
    )
    assert r.returncode == 0, r.stdout
    assert "REQ-CLU-001" in r.stdout and "PASS" in r.stdout
    assert "REQ-CAN-003" in r.stdout

#
=====
# C. SIL / HIL: one test body, two back-ends
#
=====

class SimulatedADC:
    """SIL back-end: always available."""

    def __init__(self, counts=2048):
        self.counts = counts

    def read(self):
        return self.counts

class BenchADC:
    """HIL back-end: talks to a real bench (serial / CAN-FD / NI-DAQ ...). Not available here."""

    def __init__(self):
        raise RuntimeError("no hardware bench connected") # replaced by a real driver in the lab

    def read(self): # pragma: no cover
        raise NotImplementedError

@pytest.fixture(params=["sil", pytest.param("hil", marks=pytest.mark.hil)])
def adc(request):
    return SimulatedADC() if request.param == "sil" else BenchADC()

def test_mid_scale_temperature_is_plausible_on_every_back_end(adc):
    # Same assertions on the simulator (default) and on the bench (pytest --hil)
    assert -40 < TemperatureSensor(adc).read() < 150

def test_hil_tests_are_skipped_without_the_flag(pytestconfig):
    if pytestconfig.getoption("--hil"):
        pytest.skip("running with --hil: this check only applies to the default run")
    r = subprocess.run(
        [sys.executable, "-m", "pytest", __file__, "-q", "-rs", "-p", "no:cacheprovider", "-k", "mid_scale"],
        cwd=R00T, capture_output=True, text=True,
    )
    assert "needs hardware" in r.stdout

```

Exercise skeleton — source

labs/lab13_acceptance_traceability/exercise_lab13.py

```

"""Lab 13 exercises. Run: pytest labs/lab13_acceptance_traceability/exercise_lab13.py -v"""
import pytest

# Exercise 1 - Requirements-driven development.

```

```

# docs/requirements.csv already lists REQ-TPMS-001 and REQ-TPMS-002 (tyre pressure
monitoring).
# If the instructor's solutions/ folder is absent, the meta-test
# `test_every_requirement_is_verified_by_at_least_one_test` is RED: no test verifies them
yet.
# Write tests for autotest.tpms (classify -> REQ-TPMS-001, LeakDetector -> REQ-TPMS-002)
carrying
# @pytest.mark.requirement("REQ-TPMS-00x") and watch the meta-test turn green.
# Bonus: add REQ-TPMS-003 for sensor-id validation to the CSV yourself and repeat red ->
green.
def test_new_requirement_gets_a_test():
    pytest.fail("TODO")

# Exercise 2 - Write acceptance scenario (Given/When/Then comments) for the cluster:
# "Given the engine is running and the bus is healthy, when the driver accelerates from 0 to
# 100 km/h in 10 steps, then the displayed speed never decreases." Mark it REQ-CLU-001.
def test_dashboard_never_goes_backwards_during_acceleration(bus, clock):
    pytest.fail("TODO")

# Exercise 3 - Extend `adc` fixture idea: write a `can_backend` fixture with params "virtual"
and a
# hil-marked "pcan" that is skipped by default, then run the Lab 5 smoke test on both.
def test_backend_switch():
    pytest.fail("TODO")

```

Capstone — Tyre-Pressure Monitoring bug hunt

Duration: 4–6 h (individual or pairs) · **Uses:** every technique from Labs 1–13

Scenario

You join a TPMS team. The module `src/autotest/tpms.py` has no tests. Your job: write a test suite that is **good enough to catch regressions** — the team will later "refactor" the module, and your tests are the safety net.

Specification (the only thing you may rely on)

Pressures in kPa, temperatures in °C.

ID	Rule
S1	Normalise to 20 °C: $p_{20} = p \cdot 293.15 / (273.15 + T)$
S2	Nominal pressure 230 kPa (overridable)
S3	$p_{20} < 60\%$ of nominal → CRITICAL
S4	$60\% \leq p_{20} < 80\%$ → LOW
S5	$80\% \leq p_{20} \leq 130\%$ → OK
S6	$p_{20} > 130\%$ → HIGH
S7	A drop of ≥ 20 kPa within a 60 s window (relative to the highest pressure in the window) → rapid-loss alarm
S8	Sensor ids: exactly 8 hex digits, either case; invalid → <code>ValueError</code> ; valid ids are returned upper-case
S9	Readings outside 0...700 kPa or -40...125 °C → <code>ValueError</code>

Requirements for traceability: **REQ-TPMS-001** (S3 critical), **REQ-TPMS-002** (S7 leak alarm).

Deliverables

1. `capstone/test_tpms_student.py` (replace the starter; split into several files if you like).
2. Use **at least**: parametrization with ids · boundary value analysis · a fixture · one property-based test · a `requirement` marker on the relevant tests · an equivalence-class argument in comments.
3. A short `capstone/REPORT.md` (≤ 1 page): your test strategy, what you did *not* test and why, the coverage figure, and which technique found the most.

How you are graded

```
python capstone/grade.py          # injects 17 hidden defects one at a time; you must
catch them
pytest capstone --cov=autotest.tpms --cov-branch --cov-report=term-missing
```

Criterion	Weight
Defects caught (<code>grade.py</code> score)	40 %
Suite is green on the correct code and deterministic (no sleeps, seeded randomness)	15 %
Test design quality: boundaries, classes, readable names, one reason to fail	20 %
Traceability + coverage evidence	10 %
Report	15 %

The grader prints *symptoms* of missed defects (e.g. "classification wrong exactly at the LOW/OK threshold") — never the code change — so you can iterate. Aim for 100 % (the instructor's reference suite scores 17/17).

Hints (read only if stuck)

▼ Hint 1

For each threshold in S3–S6 test just-below / on / just-above at 20 °C where the float arithmetic is exact.

▼ Hint 2

Limits in S9 are inclusive: test the limit itself *and* the next representable value beyond it, on both axes.

▼ Hint 3

For S7 think about: exactly 20, just under 20, an old peak that should have expired, the window edge at exactly 60 s, and a gentle slope that adds up to a big drop over many minutes.

▼ Hint 4

Mutation thinking: for every `<`, `<=`, constant and `max/min` in the spec, ask "which assertion fails if this is wrong?"

Starter test suite — source

capstone/test_tpms_student.py

```
"""Capstone starter - replace/extend with YOUR test suite for autotest.tpms.

Read capstone/README.md first. Grade yourself with: python capstone/grade.py
"""
import pytest
```

```

from autotest.tpms import LeakDetector, TyreStatus, classify, normalise_pressure,
validate_sensor_id

@pytest.mark.smoke
def test_nominal_tyre_is_ok():
    assert classify(230, 20) is TyreStatus.OK

# TODO 1 normalisation (S1) -> exact values, cold and hot
# TODO 2 classification (S3-S6) -> every threshold: just below / on / just above
# TODO 3 nominal override -> classify(..., nominal=250)
# TODO 4 input validation (S8, S9) -> every limit: on it (ok) and just beyond
(ValueError)
# TODO 5 leak detection (S7) -> drop sizes, window edges, recovery, falling
slowly
# TODO 6 one property-based test -> classify is monotonic in pressure
# TODO 7 mark requirement tests -> @pytest.mark.requirement("REQ-TPMS-001") / "REQ-
TPMS-002"

```

Appendix A — Cheat sheet

Task	unittest	pytest
Test discovery	<code>class X(unittest.TestCase) ,</code> methods <code>test_*</code>	any function <code>test_*</code> (classes optional, no base class)
Assertion	<code>self.assertEqual(a, b)</code> and ~30 other <code>assertXxx</code>	<code>assert a == b</code> (introspected failure message)
Floats	<code>assertAlmostEqual(a, b,</code> <code>places=7)</code>	<code>assert a == pytest.approx(b, rel=1e-6,</code> <code>abs=1e-12)</code>
Exception	<code>with self.assertRaises(E): /</code> <code>assertRaisesRegex</code>	<code>with pytest.raises(E, match="re") as</code> <code>info:</code>
Setup / teardown	<code>setUp, tearDown, setUpClass,</code> <code>tearDownClass</code>	fixtures with <code>yield</code> ; scopes <code>function/class/module/session</code>
Many inputs	<code>with self.subTest(x=x):</code> in a loop	<code>@pytest.mark.parametrize("x", [...], ids=</code> <code>[...])</code>
Skip	<code>@unittest.skip, skipIf,</code> <code>skipUnless</code>	<code>@pytest.mark.skip, skipif(cond,</code> <code>reason=...), pytest.skip()</code>
Known failure	<code>@unittest.expectedFailure</code>	<code>@pytest.mark.xfail(reason=...,</code> <code>strict=True)</code>
Temp files	<code>tempfile.TemporaryDirectory</code>	<code>tmp_path, tmp_path_factory</code>
Patch	<code>mock.patch("pkg.mod.name")</code>	<code>monkeypatch.setattr/setenv/delattr</code> (or <code>mock.patch</code>)
Capture output	<code>contextlib.redirect_stdout</code>	<code>capsys, caplog</code>
Run	<code>python -m unittest discover -v</code>	<code>pytest -v</code> (also runs unittest tests)

Everyday pytest commands

```
pytest -x                # stop at first failure
pytest --lf              # re-run only last failures
pytest -k "cruise and not hill"  # select by name expression
pytest -m "smoke"        # select by marker; -m "not slow and not performance"
pytest path/file.py::test_name[param-id]  # one test
pytest -vv --tb=short    # verbose, compact tracebacks
pytest --durations=10    # slowest tests
pytest -rs               # why were tests skipped?
pytest --pdb             # debugger at failure
pytest --cov=autotest --cov-branch --cov-report=term-missing
pytest --junitxml=report.xml
pytest --hil / --update-golden / --req-report  # this course's options
```

Choosing the right double

Double	Purpose	Example in this course
Dummy	fills a parameter, never used	listener subscribed to another id
Stub	returns canned answers	<code>StubADC(0)</code> → short-to-ground
Fake	simplified working implementation	<code>FakeClock</code> , <code>VirtualCANBus</code> , <code>SineWaveADC</code>
Spy	real object + call recording	<code>SpyADC.calls</code> , <code>SpyListener.frames</code>
Mock	verifies interactions	<code>Mock(spec=DiagnosticManager).report.assert_called_with(...)</code>

Prefer **state/outcome** assertions over interaction assertions; mock only what is slow, non-deterministic or hardware.

Test-design checklist

- One reason to fail per test; name says *what* and *under which condition*.
- Arrange – Act – Assert (or Given – When – Then).
- Boundaries: on, just below, just above. Empty, one, many. Zero, negative, NaN, huge.
- Determinism: inject clocks and RNG seeds; no `sleep`, no network, no shared state.
- Every defect fixed gets a regression test **first** (red → green).
- Tests are code: review them, refactor them, delete the ones that no longer protect anything.

Appendix B — Code under test

The complete `src/autotest` library that every lab tests. Read it only after designing tests from the specifications — that is the point of Lab 3.

init.py

`src/autotest/__init__.py`

```
"""Automotive software under test for the 'Testing with Python' course."""
```

abs.py

`src/autotest/abs.py`

```
"""ABS slip controller and a quarter-car plant for software-in-the-loop tests."""
from __future__ import annotations

import enum
from dataclasses import dataclass

G = 9.81

class Valve(enum.Enum):
    APPLY = "apply"
    HOLD = "hold"
    RELEASE = "release"

def slip_ratio(vehicle_ms: float, wheel_ms: float) -> float:
    """0 = free rolling, 1 = locked wheel. Undefined (0) at standstill."""
    if vehicle_ms <= 0.5:
        return 0.0
    return min(1.0, max(0.0, (vehicle_ms - wheel_ms) / vehicle_ms))

class ABSController:
    RELEASE_ABOVE = 0.25
    APPLY_BELOW = 0.15
    MIN_ACTIVE_SPEED = 5.0 # m/s - ABS is disabled below this

    def decide(self, vehicle_ms: float, wheel_ms: float) -> Valve:
        if vehicle_ms < self.MIN_ACTIVE_SPEED:
            return Valve.APPLY
        s = slip_ratio(vehicle_ms, wheel_ms)
        if s > self.RELEASE_ABOVE:
            return Valve.RELEASE
        if s < self.APPLY_BELOW:
            return Valve.APPLY
        return Valve.HOLD

    def tyre_mu(slip: float) -> float:
        """Simplified friction curve: peak 1.0 at 20 % slip, 0.7 when locked."""
        if slip <= 0.2:
            return slip / 0.2
        return 1.0 - 0.3 * (slip - 0.2) / 0.8
```

```

@dataclass
class QuarterCar:
    speed_ms: float = 27.8      # ~100 km/h
    wheel_ms: float = 27.8
    pressure: float = 0.0      # 0..1
    distance_m: float = 0.0
    mass: float = 400.0
    radius: float = 0.3
    inertia: float = 1.5
    max_brake_torque: float = 2000.0

    def step(self, dt: float, valve: Valve) -> None:
        rate = {Valve.APPLY: 10.0, Valve.HOLD: 0.0, Valve.RELEASE: -40.0}[valve]
        self.pressure = min(1.0, max(0.0, self.pressure + rate * dt))
        mu = tyre_mu(slip_ratio(self.speed_ms, self.wheel_ms))
        force = mu * self.mass * G
        wheel_omega = self.wheel_ms / self.radius
        brake_torque = self.pressure * self.max_brake_torque if wheel_omega > 0 else 0.0
        wheel_omega = max(0.0, wheel_omega + (force * self.radius - brake_torque) /
self.inertia * dt)
        if self.speed_ms <= 0.5 and wheel_omega * self.radius > self.speed_ms:
            wheel_omega = self.speed_ms / self.radius
        self.wheel_ms = wheel_omega * self.radius
        self.speed_ms = max(0.0, self.speed_ms - force / self.mass * dt)
        self.distance_m += self.speed_ms * dt

    def simulate_braking(use_abs: bool, dt: float = 0.001, t_max: float = 15.0) -> dict:
        car, ctrl = QuarterCar(), ABSController()
        t, peak_slip, locked_time = 0.0, 0.0, 0.0
        while car.speed_ms > 0.5 and t < t_max:
            valve = ctrl.decide(car.speed_ms, car.wheel_ms) if use_abs else Valve.APPLY
            car.step(dt, valve)
            if car.speed_ms >= ABSController.MIN_ACTIVE_SPEED: # below this ABS is off by design
                s = slip_ratio(car.speed_ms, car.wheel_ms)
                peak_slip = max(peak_slip, s)
                if s > 0.95:
                    locked_time += dt
            t += dt
        return {"distance_m": car.distance_m, "time_s": t, "peak_slip": peak_slip, "locked_s":
locked_time}

```

bms.py

src/autotest/bms.py

```

"""Battery management: charge-current derating, protection and SOC."""
from __future__ import annotations

import enum

class BMSFault(enum.Enum):
    NONE = "none"
    OVERVOLTAGE = "overvoltage"
    UNDERVOLTAGE = "undervoltage"
    OVERTEMPERATURE = "overtemperature"
    UNDERTEMPERATURE = "undertemperature"

    def max_charge_current(soc: float, temp_c: float, rated_a: float = 100.0) -> float:
        """Allowed charge current (A).

```

```

Decision table (see Lab 3):
temp < 0 or temp > 45    -> 0 A    (lithium plating / thermal risk)
0 <= temp < 10          -> 25 % of rated
10 <= temp <= 45        -> 100 % of rated
soc >= 100              -> 0 A
soc > 80                -> taper linearly from the temperature limit to 0 at 100 %
"""

if not 0 <= soc <= 100:
    raise ValueError("soc must be within 0..100")
if temp_c < 0 or temp_c > 45:
    return 0.0
limit = rated_a * (0.25 if temp_c < 10 else 1.0)
if soc >= 100:
    return 0.0
if soc > 80:
    limit *= (100 - soc) / 20
return limit

class BatteryManagementSystem:
    CELL_V_MAX = 4.20
    CELL_V_MIN = 3.00
    TEMP_MAX = 60.0
    TEMP_MIN = -20.0

    def __init__(self, capacity_ah: float = 50.0, soc: float = 50.0):
        if capacity_ah <= 0:
            raise ValueError("capacity must be positive")
        self.capacity_ah = capacity_ah
        self.soc = soc

    def evaluate(self, cell_voltages: list[float], temp_c: float) -> BMSFault:
        if not cell_voltages:
            raise ValueError("no cell data")
        if max(cell_voltages) > self.CELL_V_MAX:
            return BMSFault.OVERVOLTAGE
        if min(cell_voltages) < self.CELL_V_MIN:
            return BMSFault.UNDERVOLTAGE
        if temp_c > self.TEMP_MAX:
            return BMSFault.OVERTEMPERATURE
        if temp_c < self.TEMP_MIN:
            return BMSFault.UNDERTEMPERATURE
        return BMSFault.NONE

    def update_soc(self, current_a: float, dt_s: float) -> float:
        """Coulomb counting. Positive current charges the pack."""
        if dt_s < 0:
            raise ValueError("dt must be >= 0")
        delta = current_a * dt_s / 3600 / self.capacity_ah * 100
        self.soc = min(100.0, max(0.0, self.soc + delta))
        return self.soc

```

bus.py

src/autotest/bus.py

```

"""A virtual CAN bus with fault-injection hooks, used for integration tests."""
from __future__ import annotations

from typing import Callable, Optional

from .can import CANFrame

Listener = Callable[[CANFrame], None]
FaultHook = Callable[[CANFrame], Optional[CANFrame]]

```

```

class VirtualCANBus:
    def __init__(self) -> None:
        self._listeners: list[tuple[Listener, Optional[set[int]]]] = []
        self._fault_hooks: list[FaultHook] = []
        self.log: list[CANFrame] = []

    def subscribe(self, listener: Listener, ids: set[int] | None = None) -> None:
        self._listeners.append((listener, ids))

    def add_fault(self, hook: FaultHook) -> None:
        """A hook may return a modified frame, or None to drop it."""
        self._fault_hooks.append(hook)

    def clear_faults(self) -> None:
        self._fault_hooks.clear()

    def send(self, frame: CANFrame) -> bool:
        self.log.append(frame)
        current: Optional[CANFrame] = frame
        for hook in self._fault_hooks:
            current = hook(current) # type: ignore[arg-type]
            if current is None:
                return False
        for listener, ids in self._listeners:
            if ids is None or current.arbitration_id in ids:
                listener(current)
        return True

```

can.py

src/autotest/can.py

```

"""CAN frames, signal packing and end-to-end (E2E) protection."""
from __future__ import annotations

import enum
from dataclasses import dataclass, field

class E2EStatus(enum.Enum):
    OK = "ok"
    TOO_SHORT = "too_short"
    CRC_ERROR = "crc_error"
    COUNTER_ERROR = "counter_error"

def crc8(data: bytes, poly: int = 0x1D, init: int = 0xFF, xor_out: int = 0xFF) -> int:
    """CRC-8 SAE J1850 (the AUTOSAR E2E profile 1 polynomial)."""
    crc = init
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = ((crc << 1) ^ poly) & 0xFF if crc & 0x80 else (crc << 1) & 0xFF
    return crc ^ xor_out

@dataclass(frozen=True)
class CANFrame:
    arbitration_id: int
    data: bytes = b""
    is_extended: bool = False

    def __post_init__(self) -> None:

```

```

        limit = 0x1FFFFFFF if self.is_extended else 0x7FF
        if not 0 <= self.arbitration_id <= limit:
            raise ValueError(f"arbitration id 0x{self.arbitration_id:X} out of range")
        if len(self.data) > 8:
            raise ValueError("classic CAN carries at most 8 data bytes")

    @property
    def dlc(self) -> int:
        return len(self.data)

@dataclass(frozen=True)
class Signal:
    """A physical value packed into a frame (Intel / little-endian bit order)."""

    name: str
    start_bit: int
    length: int
    factor: float = 1.0
    offset: float = 0.0
    minimum: float | None = None
    maximum: float | None = None
    signed: bool = False

    def __post_init__(self) -> None:
        if not 1 <= self.length <= 64 or self.start_bit < 0:
            raise ValueError("invalid signal geometry")
        if self.factor == 0:
            raise ValueError("factor must not be zero")

    def raw_range(self) -> tuple[int, int]:
        if self.signed:
            return -(1 << (self.length - 1)), (1 << (self.length - 1)) - 1
        return 0, (1 << self.length) - 1

    def to_raw(self, physical: float) -> int:
        if self.minimum is not None and physical < self.minimum:
            raise ValueError(f"{self.name}: {physical} below minimum {self.minimum}")
        if self.maximum is not None and physical > self.maximum:
            raise ValueError(f"{self.name}: {physical} above maximum {self.maximum}")
        raw = round((physical - self.offset) / self.factor)
        low, high = self.raw_range()
        if not low <= raw <= high:
            raise ValueError(f"{self.name}: raw value {raw} does not fit {self.length} bits")
        return raw

    def to_physical(self, raw: int) -> float:
        return raw * self.factor + self.offset

@dataclass(frozen=True)
class CANMessage:
    """Message definition: a frame id, a payload length and its signals."""

    frame_id: int
    name: str
    dlc: int
    signals: tuple[Signal, ...] = field(default_factory=tuple)

    def __post_init__(self) -> None:
        if not 0 <= self.dlc <= 8:
            raise ValueError("dlc must be 0..8")
        for s in self.signals:
            if s.start_bit + s.length > self.dlc * 8:
                raise ValueError(f"signal {s.name} does not fit in {self.dlc} bytes")

    def encode(self, values: dict[str, float]) -> bytes:
        unknown = set(values) - {s.name for s in self.signals}

```

```

        if unknown:
            raise KeyError(f"unknown signals: {sorted(unknown)}")
        word = 0
        for s in self.signals:
            raw = s.to_raw(values.get(s.name, s.offset))
            word |= (raw & ((1 << s.length) - 1)) << s.start_bit
        return word.to_bytes(self.dlc, "little")

    def decode(self, payload: bytes) -> dict[str, float]:
        if len(payload) != self.dlc:
            raise ValueError(f"{self.name}: expected {self.dlc} bytes, got {len(payload)}")
        word = int.from_bytes(payload, "little")
        out: dict[str, float] = {}
        for s in self.signals:
            raw = (word >> s.start_bit) & ((1 << s.length) - 1)
            if s.signed and raw >= 1 << (s.length - 1):
                raw -= 1 << s.length
            out[s.name] = s.to_physical(raw)
        return out

# ---- End-to-end protection: [crc8][counter][payload...] -----

    def e2e_protect(payload: bytes, counter: int) -> bytes:
        counter &= 0x0F
        body = bytes([counter]) + payload
        return bytes([crc8(body)]) + body

    def e2e_check(data: bytes, expected_counter: int | None = None) -> tuple[E2EStatus, bytes]:
        """Return (status, payload). Payload is empty unless status is OK."""
        if len(data) < 2:
            return E2EStatus.TOO_SHORT, b""
        if crc8(data[1:]) != data[0]:
            return E2EStatus.CRC_ERROR, b""
        if expected_counter is not None and data[1] != (expected_counter & 0x0F):
            return E2EStatus.COUNTER_ERROR, b""
        return E2EStatus.OK, data[2:]

```

clock.py

src/autotest/clock.py

```

"""Injectable clocks: production code takes a clock, tests pass a FakeClock."""
import time

class SystemClock:
    def now(self) -> float:
        return time.monotonic()

class FakeClock:
    """Deterministic clock - the key trick for testing timeouts without sleeping."""

    def __init__(self, start: float = 0.0):
        self._t = start

    def now(self) -> float:
        return self._t

    def advance(self, seconds: float) -> None:
        if seconds < 0:

```

```
        raise ValueError("time cannot go backwards")
    self._t += seconds
```

cruise.py

src/autotest/cruise.py

```
"""Adaptive-free cruise control: mode state machine plus a PI speed controller."""
from __future__ import annotations

import enum

class CruiseState(enum.Enum):
    OFF = "off"
    STANDBY = "standby"
    ACTIVE = "active"
    OVERRIDE = "override" # driver is pressing the accelerator

class CruiseController:
    MIN_SET_SPEED = 30.0 # km/h
    MAX_SET_SPEED = 180.0 # km/h

    def __init__(self, kp: float = 0.05, ki: float = 0.01):
        self.kp, self.ki = kp, ki
        self.state = CruiseState.OFF
        self.target: float | None = None
        self.saved_target: float | None = None
        self._integral = 0.0

    # --- driver inputs -----
    def power_on(self) -> None:
        if self.state is CruiseState.OFF:
            self.state = CruiseState.STANDBY

    def power_off(self) -> None:
        self.state = CruiseState.OFF
        self.target = self.saved_target = None
        self._integral = 0.0

    def set(self, speed_kmh: float) -> bool:
        """Engage / change the set speed. Returns False if the request is refused."""
        if self.state is CruiseState.OFF:
            return False
        if not self.MIN_SET_SPEED <= speed_kmh <= self.MAX_SET_SPEED:
            return False
        self.target = speed_kmh
        self.saved_target = speed_kmh
        self.state = CruiseState.ACTIVE
        self._integral = 0.0
        return True

    def resume(self, current_speed_kmh: float) -> bool:
        if self.state is not CruiseState.STANDBY or self.saved_target is None:
            return False
        if current_speed_kmh < self.MIN_SET_SPEED:
            return False
        return self.set(self.saved_target)

    def cancel(self) -> None:
        if self.state in (CruiseState.ACTIVE, CruiseState.OVERRIDE):
            self.saved_target = self.target
            self.target = None
            self.state = CruiseState.STANDBY
```

```

        self._integral = 0.0

    brake = cancel # pressing the brake pedal always cancels

    def accelerator(self, pressed: bool) -> None:
        if pressed and self.state is CruiseState.ACTIVE:
            self.state = CruiseState.OVERRIDE
        elif not pressed and self.state is CruiseState.OVERRIDE:
            self.state = CruiseState.ACTIVE
            self._integral = 0.0

    # --- control law -----
    def control(self, speed_kmh: float, dt: float) -> float:
        """Throttle request 0..1. Zero unless ACTIVE."""
        if self.state is not CruiseState.ACTIVE or self.target is None:
            return 0.0
        error = self.target - speed_kmh
        unsat = self.kp * error + self.ki * (self._integral + error * dt)
        out = min(1.0, max(0.0, unsat))
        if out == unsat: # anti-windup: only integrate while not saturated
            self._integral += error * dt
        return out

```

diagnostics.py

src/autotest/diagnostics.py

```

"""Diagnostic trouble codes (DTC) with debouncing and a UDS-lite request handler."""
from __future__ import annotations

from dataclasses import dataclass

NRC_SERVICE_NOT_SUPPORTED = 0x11
NRC_INCORRECT_LENGTH = 0x13
NRC_REQUEST_OUT_OF_RANGE = 0x31

@dataclass
class _DTC:
    fail_count: int = 0
    pass_count: int = 0
    active: bool = False
    stored: bool = False

class DiagnosticManager:
    """A DTC is confirmed after `fail_threshold` consecutive failures and
    stops being active after `heal_threshold` consecutive passes (it stays stored)."""

    VIN = b"WAUZZZ8K9BA123456"

    def __init__(self, fail_threshold: int = 3, heal_threshold: int = 5):
        if fail_threshold < 1 or heal_threshold < 1:
            raise ValueError("thresholds must be >= 1")
        self.fail_threshold = fail_threshold
        self.heal_threshold = heal_threshold
        self._dtcs: dict[str, _DTC] = {}

    @staticmethod
    def _validate(code: str) -> None:
        if len(code) != 5 or code[0] not in "PCBU" or not all(c in "0123456789ABCDEF" for c
in code[1:]):
            raise ValueError(f"malformed DTC {code!r}")

    def report(self, code: str, failed: bool) -> None:

```

```

    self._validate(code)
    d = self._dtcs.setdefault(code, _DTC())
    if failed:
        d.fail_count, d.pass_count = d.fail_count + 1, 0
        if d.fail_count >= self.fail_threshold:
            d.active = d.stored = True
    else:
        d.pass_count, d.fail_count = d.pass_count + 1, 0
        if d.pass_count >= self.heal_threshold:
            d.active = False

def active_codes(self) -> list[str]:
    return sorted(c for c, d in self._dtcs.items() if d.active)

def stored_codes(self) -> list[str]:
    return sorted(c for c, d in self._dtcs.items() if d.stored)

def clear_all(self) -> None:
    self._dtcs.clear()

def export_report(self) -> str:
    lines = ["DTC REPORT", "====="]
    for code in sorted(self._dtcs):
        d = self._dtcs[code]
        lines.append(f"{code} active={d.active} stored={d.stored}")
    if len(lines) == 2:
        lines.append("(no trouble codes)")
    return "\n".join(lines) + "\n"

# ---- UDS-lite (ISO 14229 subset) -----
@staticmethod
def _negative(service: int, nrc: int) -> bytes:
    return bytes([0x7F, service, nrc])

def handle_request(self, request: bytes) -> bytes:
    if not request:
        return self._negative(0x00, NRC_INCORRECT_LENGTH)
    sid = request[0]
    if sid == 0x19: # ReadDTCInformation, sub-function 0x02 = by status mask
        if len(request) != 3:
            return self._negative(sid, NRC_INCORRECT_LENGTH)
        if request[1] != 0x02:
            return self._negative(sid, NRC_REQUEST_OUT_OF_RANGE)
        body = b"".join(c.encode() + b";" for c in self.stored_codes())
        return bytes([0x59, 0x02]) + body
    if sid == 0x14: # ClearDiagnosticInformation
        self.clear_all()
        return bytes([0x54])
    if sid == 0x22: # ReadDataByIdentifier
        if len(request) != 3:
            return self._negative(sid, NRC_INCORRECT_LENGTH)
        if request[1:3] != b"\xF1\x90":
            return self._negative(sid, NRC_REQUEST_OUT_OF_RANGE)
        return bytes([0x62, 0xF1, 0x90]) + self.VIN
    return self._negative(sid, NRC_SERVICE_NOT_SUPPORTED)

```

ecu.py

src/autotest/ecu.py

```

"""Two cooperating ECUs: a wheel-speed sender and an instrument cluster receiver."""
from __future__ import annotations

from .bus import VirtualCANBus
from .can import CANFrame, CANMessage, E2EStatus, Signal, e2e_check, e2e_protect

```

```

from .clock import FakeClock, SystemClock
from .diagnostics import DiagnosticManager

WHEEL_SPEED_MSG = CANMessage(
    frame_id=0x1A0,
    name="WheelSpeed",
    dlc=3,
    signals=(
        Signal("speed_kmh", 0, 16, factor=0.01, minimum=0.0, maximum=300.0),
        Signal("valid", 16, 1),
    ),
)

DTC_COMM_LOST = "U0121"
DTC_E2E_FAIL = "U0100"

class WheelSpeedECU:
    def __init__(self, bus: VirtualCANBus):
        self.bus = bus
        self._counter = 0

    def send_speed(self, speed_kmh: float) -> bool:
        payload = WHEEL_SPEED_MSG.encode({"speed_kmh": speed_kmh, "valid": 1})
        data = e2e_protect(payload, self._counter)
        self._counter = (self._counter + 1) & 0x0F
        return self.bus.send(CANFrame(WHEEL_SPEED_MSG.frame_id, data))

class InstrumentCluster:
    TIMEOUT_S = 0.1

    def __init__(self, bus: VirtualCANBus, clock: FakeClock | SystemClock | None = None,
                 diag: DiagnosticManager | None = None):
        self.clock = clock or SystemClock()
        self.diag = diag or DiagnosticManager(fail_threshold=2)
        self._speed: float | None = None
        self._last_rx = self.clock.now()
        self._expected_counter: int | None = None
        bus.subscribe(self._on_frame, ids={WHEEL_SPEED_MSG.frame_id})

    def _on_frame(self, frame: CANFrame) -> None:
        status, payload = e2e_check(frame.data, self._expected_counter)
        if status is not E2EStatus.OK:
            self.diag.report(DTC_E2E_FAIL, failed=True)
            if status is E2EStatus.COUNTER_ERROR: # resynchronise after a lost frame
                self._expected_counter = (frame.data[1] + 1) & 0x0F
            elif self._expected_counter is not None:
                # BUG-130: a corrupted frame still occupies one slot in the sequence, so the
                # next good frame must not be rejected as a counter error as well.
                self._expected_counter = (self._expected_counter + 1) & 0x0F
            return
        self.diag.report(DTC_E2E_FAIL, failed=False)
        signals = WHEEL_SPEED_MSG.decode(payload)
        self._speed = signals["speed_kmh"] if signals["valid"] else None
        self._last_rx = self.clock.now()
        self._expected_counter = (frame.data[1] + 1) & 0x0F

    def displayed_speed(self) -> str:
        if self.clock.now() - self._last_rx > self.TIMEOUT_S:
            self.diag.report(DTC_COMM_LOST, failed=True)
            return "---"
        self.diag.report(DTC_COMM_LOST, failed=False)
        return "---" if self._speed is None else f"{self._speed:.0f}"

```

sensors.py

src/autotest/sensors.py

```
"""Sensor conversions and a plausibility-checked temperature sensor."""
from __future__ import annotations

from collections import deque
from typing import Protocol

class SensorFault(Exception):
    """Raised when a raw reading is physically implausible."""

def kmh_to_ms(kmh: float) -> float:
    return kmh / 3.6

def ms_to_kmh(ms: float) -> float:
    return ms * 3.6

def celsius_to_fahrenheit(c: float) -> float:
    return c * 9 / 5 + 32

def wheel_speed_kmh(pulses: int, pulses_per_rev: int, circumference_m: float, interval_s: float) -> float:
    if pulses < 0 or pulses_per_rev <= 0 or circumference_m <= 0 or interval_s <= 0:
        raise ValueError("invalid wheel speed parameters")
    revs_per_s = pulses / pulses_per_rev / interval_s
    return ms_to_kmh(revs_per_s * circumference_m)

class ADC(Protocol):
    """Hardware abstraction: real boards, simulators and mocks all fit this."""

    def read(self) -> int: ...

class TemperatureSensor:
    """12-bit ADC mapped linearly to -40..150 degC. Rail values mean a wiring fault."""

    ADC_MAX = 4095
    T_MIN, T_MAX = -40.0, 150.0

    def __init__(self, adc: ADC, filter_len: int = 4):
        if filter_len < 1:
            raise ValueError("filter_len must be >= 1")
        self._adc = adc
        self._window: deque[float] = deque(maxlen=filter_len)

    def read(self) -> float:
        counts = self._adc.read()
        if counts <= 0:
            raise SensorFault("short to ground")
        if counts >= self.ADC_MAX:
            raise SensorFault("open circuit / short to supply")
        return self.T_MIN + (self.T_MAX - self.T_MIN) * counts / self.ADC_MAX

    def read_filtered(self) -> float:
        self._window.append(self.read())
        return sum(self._window) / len(self._window)
```

tpms.py

src/autotest/tpms.py

```
"""Tyre pressure monitoring - the capstone system under test.

Specification (all pressures in kPa, temperatures in degC)
S1 Pressure is normalised to 20 degC:  $p_{20} = p * 293.15 / (273.15 + temp)$ 
S2 Nominal pressure is 230 kPa.
S3  $p_{20} < 60\%$  of nominal -> CRITICAL
S4  $60\% \leq p_{20} < 80\%$  of nominal -> LOW
S5  $80\% \leq p_{20} \leq 130\%$  of nominal -> OK
S6  $p_{20} > 130\%$  of nominal -> HIGH
S7 A drop of  $\geq 20$  kPa within a 60 s window -> rapid loss (leak) alarm
S8 Sensor ids are exactly 8 hex digits (either case); anything else is rejected with
   ValueError. Valid ids are returned in UPPER case.
S9 Readings outside 0..700 kPa or -40..125 degC are rejected with ValueError.
"""

from __future__ import annotations

import enum
import re
from collections import deque

NOMINAL_KPA = 230.0

class TyreStatus(enum.Enum):
    CRITICAL = "critical"
    LOW = "low"
    OK = "ok"
    HIGH = "high"

def normalise_pressure(kpa: float, temp_c: float) -> float:
    return kpa * 293.15 / (273.15 + temp_c)

def classify(kpa: float, temp_c: float, nominal: float = NOMINAL_KPA) -> TyreStatus:
    if not 0 <= kpa <= 700 or not -40 <= temp_c <= 125:
        raise ValueError("reading out of sensor range")
    p20 = normalise_pressure(kpa, temp_c)
    if p20 < 0.6 * nominal:
        return TyreStatus.CRITICAL
    if p20 < 0.8 * nominal:
        return TyreStatus.LOW
    if p20 <= 1.3 * nominal:
        return TyreStatus.OK
    return TyreStatus.HIGH

_SENSOR_ID = re.compile(r"[0-9A-Fa-f]{8}")

def validate_sensor_id(sensor_id: str) -> str:
    if not _SENSOR_ID.fullmatch(sensor_id):
        raise ValueError(f"bad sensor id {sensor_id!r}")
    return sensor_id.upper()

class LeakDetector:
    WINDOW_S = 60.0
    DROP_KPA = 20.0

    def __init__(self) -> None:
        self._samples: deque[tuple[float, float]] = deque()
```

```

def add(self, t: float, kpa: float) -> bool:
    """Record a sample, return True when a rapid loss is detected."""
    self._samples.append((t, kpa))
    while self._samples and t - self._samples[0][0] > self.WINDOW_S:
        self._samples.popleft()
    highest = max(p for _, p in self._samples)
    return highest - kpa >= self.DROP_KPA

```

vehicle.py

src/autotest/vehicle.py

```

"""Longitudinal vehicle model (point mass + drag + rolling resistance + grade)."""
from __future__ import annotations

import math
from dataclasses import dataclass

from .cruise import CruiseController

G = 9.81

@dataclass
class Vehicle:
    speed_ms: float = 0.0
    mass: float = 1500.0
    max_drive_force: float = 2500.0  # N at full throttle
    cd_a: float = 0.66  # drag coefficient x frontal area (m^2)
    crr: float = 0.01  # rolling resistance coefficient
    rho: float = 1.2  # air density

    def step(self, throttle: float, dt: float, grade: float = 0.0, brake_force: float = 0.0)
-> float:
        throttle = min(1.0, max(0.0, throttle))
        drag = 0.5 * self.rho * self.cd_a * self.speed_ms ** 2
        rolling = self.crr * self.mass * G if self.speed_ms > 0 else 0.0
        slope = self.mass * G * math.sin(math.atan(grade))
        net = throttle * self.max_drive_force - drag - rolling - slope - brake_force
        self.speed_ms = max(0.0, self.speed_ms + net / self.mass * dt)
        return self.speed_ms * 3.6

    @property
    def speed_kmh(self) -> float:
        return self.speed_ms * 3.6

def run_cruise(cc: CruiseController, start_kmh: float, duration_s: float, dt: float = 0.1,
               grade_profile=lambda t: 0.0, events=None):
    """Closed-loop simulation. `events` maps time (s) -> callable(cc, vehicle)."""
    car = Vehicle(speed_ms=start_kmh / 3.6)
    events = dict(events or {})
    trace = []
    for k in range(int(duration_s / dt)):
        t = round(k * dt, 6)
        if t in events:
            events[t](cc, car)
        throttle = cc.control(car.speed_kmh, dt)
        car.step(throttle, dt, grade_profile(t))
        trace.append((t, car.speed_kmh, throttle))
    return trace

```

Appendix C — Course tooling

Shared fixtures and options

conftest.py

```
"""Shared fixtures and a tiny requirement-traceability plugin for the whole course."""
from __future__ import annotations

from collections import defaultdict
from pathlib import Path

import pytest

from autotest.bus import VirtualCANBus
from autotest.clock import FakeClock

def pytest_addoption(parser):
    g = parser.getgroup("autotest course")
    g.addoption("--hil", action="store_true", help="run tests marked 'hil' (needs hardware)")
    g.addoption("--update-golden", action="store_true", help="rewrite golden files (Lab 9)")
    g.addoption("--req-report", action="store_true", help="print requirement traceability
matrix (Lab 13)")

def pytest_collection_modifyitems(config, items):
    if not config.getoption("--hil"):
        skip_hil = pytest.mark.skip(reason="needs hardware: run with --hil")
        for item in items:
            if "hil" in item.keywords:
                item.add_marker(skip_hil)

# ---- shared fixtures -----
@pytest.fixture
def clock() -> FakeClock:
    return FakeClock()

@pytest.fixture
def bus() -> VirtualCANBus:
    return VirtualCANBus()

@pytest.fixture
def golden(request):
    """Compare text with a stored golden file; `--update-golden` rewrites it."""
    update = request.config.getoption("--update-golden")
    base = Path(request.node.fspath).parent / "golden"

    def check(name: str, actual: str) -> None:
        path = base / name
        if update or not path.exists():
            base.mkdir(exist_ok=True)
            path.write_text(actual)
        if not update:
            pytest.fail(f"golden file {path} did not exist - created it; re-run to
verify")

    return

    assert actual == path.read_text(), f"output differs from {path.name}; inspect, then -
-update-golden"
```

```

        return check

# ---- requirement traceability -----
_trace: dict[str, list[tuple[str, str]]] = defaultdict(list)

def pytest_runtest_logreport(report):
    if report.when == "call":
        for req in getattr(report, "_reqs", []):
            _trace[req].append((report.nodeid, report.outcome))

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()
    rep._reqs = [m.args[0] for m in item.iter_markers("requirement") if m.args]

def pytest_terminal_summary(terminalreporter, config):
    if not config.getoption("--req-report"):
        return
    tr = terminalreporter
    tr.section("requirement traceability")
    for req in sorted(_trace):
        results = _trace[req]
        verdict = "PASS" if all(o == "passed" for _, o in results) else "FAIL"
        tr.write_line(f"{req:<14} {verdict} ({len(results)} tests)")
        for nodeid, outcome in results:
            tr.write_line(f"    {outcome:<7} {nodeid}")

```

Mutation tester (Lab 12)

tools/mini_mutate.py

```

#!/usr/bin/env python3
"""mini_mutate - a ~120 line mutation tester, small enough to read in class.

Idea: coverage tells you which lines RAN; mutation testing tells you whether your
assertions would NOTICE if the line were wrong. We make one small change (a "mutant")
to the code under test, run the tests, and expect them to FAIL ("kill" the mutant).
A mutant that survives points at a missing or weak assertion.

Example
    python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current \
        --tests labs/lab03_black_box/test_lab03_black_box.py
"""
from __future__ import annotations

import argparse
import ast
import copy
import os
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

SWAPS = {
    ast.Lt: ast.LtE, ast.LtE: ast.Lt, ast.Gt: ast.GtE, ast.GtE: ast.Gt,
    ast.Eq: ast.NotEq, ast.NotEq: ast.Eq,
    ast.Add: ast.Sub, ast.Sub: ast.Add, ast.Mult: ast.Div, ast.Div: ast.Mult,
    ast.And: ast.Or, ast.Or: ast.And,

```

```

}

class Mutator(ast.NodeTransformer):
    """Visits every mutable site in order; mutates only the one whose index == target."""

    def __init__(self, target: int, function: str | None):
        self.target, self.function = target, function
        self.count, self.sites, self._inside = 0, [], function is None

    def visit_FunctionDef(self, node):
        was = self._inside
        if self.function and node.name == self.function:
            self._inside = True
        self.generic_visit(node)
        self._inside = was
        return node

    def _site(self, node, description):
        index, self.count = self.count, self.count + 1
        self.sites.append((node.lineno, description))
        return index == self.target

    def visit_Compare(self, node):
        self.generic_visit(node)
        if self._inside:
            for i, op in enumerate(node.ops):
                if type(op) in SWAPS and self._site(node, f"{type(op).__name__} -> {SWAPS[type(op)].__name__}"):
                    node.ops[i] = SWAPS[type(op)]()
            return node

    def visit_BinOp(self, node):
        self.generic_visit(node)
        if self._inside and type(node.op) in SWAPS and self._site(node, f"{type(node.op).__name__} -> {SWAPS[type(node.op)].__name__}"):
            node.op = SWAPS[type(node.op)]()
        return node

    def visit_BoolOp(self, node):
        self.generic_visit(node)
        if self._inside and self._site(node, f"{type(node.op).__name__} -> {SWAPS[type(node.op)].__name__}"):
            node.op = SWAPS[type(node.op)]()
        return node

    def visit_Constant(self, node):
        if self._inside and isinstance(node.value, (int, float)) and not isinstance(node.value, bool):
            if self._site(node, f"constant {node.value!r} -> {node.value + 1!r}"):
                return ast.copy_location(ast.Constant(node.value + 1), node)
        return node

    def run_tests(workdir: Path, tests: list[str], timeout: int) -> bool:
        """True when the test run PASSED."""
        # Without this, two same-sized mutants written within one second would reuse the previous
        # mutant's stale .pyc (bytecode is validated by source mtime in whole seconds + size).
        env = {**os.environ, "PYTHONDONTWRITEBYTECODE": "1"}
        try:
            r = subprocess.run([sys.executable, "-m", "pytest", "-x", "-q", "-p",
                               "no:cacheprovider", *tests],
                               cwd=workdir, capture_output=True, timeout=timeout, env=env)
        except subprocess.TimeoutExpired:
            return False # an infinite loop counts as detected
        return r.returncode == 0

```

```

def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__,
    formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--target", required=True, help="python file to mutate (relative to repo
    root)")
    ap.add_argument("--function", help="only mutate inside this function")
    ap.add_argument("--tests", nargs="+", required=True, help="test files / dirs to run")
    ap.add_argument("--timeout", type=int, default=60)
    ap.add_argument("--list", action="store_true", help="only list mutation sites")
    args = ap.parse_args()

    root = Path(__file__).resolve().parents[1]
    target = root / args.target
    tree = ast.parse(target.read_text())
    probe = Mutator(-1, args.function)
    probe.visit(copy.deepcopy(tree))
    print(f"{len(probe.sites)} mutants in {args.target}" + (f"::{args.function}" if
    args.function else ""))
    if args.list:
        for i, (line, desc) in enumerate(probe.sites):
            print(f" #{i:02d} line {line}: {desc}")
        return 0

    with tempfile.TemporaryDirectory() as tmp:
        work = Path(tmp) / "proj"
        shutil.copytree(root, work, ignore=shutil.ignore_patterns(
            ".git", "__pycache__", ".pytest_cache", ".hypothesis", "htmlcov", ".coverage"))
        mutated_file = work / args.target
        if not run_tests(work, args.tests, args.timeout):
            print("The unmutated code already FAILS the tests - fix that first.")
            return 2
        survivors = []
        for i, (line, desc) in enumerate(probe.sites):
            m = Mutator(i, args.function)
            mutated_file.write_text(ast.unparse(m.visit(copy.deepcopy(tree))))
            killed = not run_tests(work, args.tests, args.timeout)
            print(f" #{i:02d} line {line:>3} {desc:<28} {'killed' if killed else
'SURVIVED'}")
            if not killed:
                survivors.append((i, line, desc))
        total = len(probe.sites)
        score = 100 * (total - len(survivors)) / total if total else 100.0
        print(f"\nmutation score: {total - len(survivors)}/{total} = {score:.0f} %")
        for i, line, desc in survivors:
            print(f" survivor #{i:02d} (line {line}): {desc} -> which assertion should have
failed?")

        return 0 if not survivors else 1

if __name__ == "__main__":
    sys.exit(main())

```

Requirements used for traceability (Lab 13)

docs/requirements.csv

```

id,component,text
REQ-CAN-001,CAN,A frame shall reject arbitration ids outside the 11-bit (standard) or 29-bit
(extended) range.
REQ-CAN-002,CAN,Signal encode followed by decode shall reproduce the physical value within one
resolution step.
REQ-CAN-003,CAN,Receivers shall reject frames whose CRC or alive counter is invalid.
REQ-BMS-001,BMS,Charging shall be inhibited below 0 degC and above 45 degC.
REQ-BMS-002,BMS,Charge current shall taper linearly to zero between 80 % and 100 % SOC.
REQ-BMS-003,BMS,Any cell above 4.20 V shall raise an overvoltage fault.

```

REQ-CC-001,Cruise,Cruise control shall only engage between 30 and 180 km/h.
REQ-CC-002,Cruise,Pressing the brake shall cancel cruise control immediately.
REQ-CC-003,Cruise,Throttle output shall be zero unless cruise control is active.
REQ-ABS-001,ABS,With ABS the wheel shall not lock while vehicle speed is above 5 m/s.
REQ-ABS-002,ABS,ABS shall shorten the stopping distance compared with locked-wheel braking on the reference surface.
REQ-DIAG-001,Diagnostics,A DTC shall be confirmed only after N consecutive failed debounce cycles.
REQ-DIAG-002,Diagnostics,Unsupported diagnostic services shall be answered with negative response code 0x11.
REQ-CLU-001,Cluster,The cluster shall display "--" when no valid speed frame arrived for 100 ms.
REQ-TPMS-001,TPMS,A tyre below 60 % of nominal pressure (normalised to 20 degC) shall be classified CRITICAL.
REQ-TPMS-002,TPMS,A pressure drop of 20 kPa or more within a 60 s window shall raise a rapid-loss alarm.